BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA
ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

BREAKING THE FOURTH WALL GAME

HRA S PRVKY BOŘENÍ ČTVRTÉ STĚNY

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **LUKÁŠ PÍŠEK**
AUTOR PRÁCE

SUPERVISOR **Ing. TOMÁŠ MILET, Ph.D.**
VEDOUCÍ PRÁCE

BRNO 2026

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

Bachelor's Thesis Assignment

169406

Institut: Department of Computer Graphics and Multimedia (DCGM)
Student: **Píšek Lukáš**
Programme: Information Technology
Title: **Breaking the Fourth Wall Game.**
Category: Computer Graphics
Academic year: 2025/26

Assignment:

1. Study the Unity engine, game development, and the design of games that break the fourth wall. Study detective work.
2. Design suitable game mechanics that break the fourth wall and affect the player's computer. Focus on unconventional gameplay elements. Set the game in a detective investigation environment.
3. Implement the designed game.
4. Evaluate the game with users.
5. Write down conclusions and findings. Create a demonstration video.

Literature:

- Adams, Ernest, and Joris Dormans. Game mechanics: advanced game design. New Riders, 2012. ISBN 0321820274, 9780321820273
- Buckland, Mat. Programming Game AI by Example 1st Edition. Jones & Bartlett Learning, 2004. ISBN-10 1556220782.
- Dunn, Fletcher. 3D Math Primer for Graphics and Game Development 2nd Edition. A K Peters/CRC Press, 2011. ISBN-10 1568817231.
- Ernest Adams. Fundamentals of Game Design 3rd Edition. New Riders 2013. ISBN-100321929675.
- Jeannie Novak, Game Development Essentials: An Introduction, Cengage Learning, 2011, ISBN 1111307652, počet stran: 510.
- Jeremy Gibson Bond. Introduction to Game Design, Prototyping, and Development: From Concept to Playable Game with Unity and C#. Addison-Wesley Professional 2017. ISBN-100134659864.
- Nystrom, Robert. 2014. Game Programming Patterns. Genever Benning; 1st edition (November 2, 2014). ISBN-10 0990582906. ISBN-13 978-0990582908.
- Steve Rabin. Game AI Pro – Online Edition 2021.

Requirements for the semestral defence:
First two points and the basis of the application.

Detailed formal requirements can be found at https://www.fit.vut.cz/study/theses/

Supervisor: **Milet Tomáš, Ing., Ph.D.**
Head of Department: Černocký Jan, prof. Dr. Ing.
Beginning of work: 1.11.2025
Submission deadline: 13.5.2026
Approval date: 26.3.2026

Abstract

This thesis focuses on the design and development of a detective horror game titled Kernel_Panic, which utilizes the innovative element of breaking the fourth wall. Within the story, the player assumes the role of a detective at a police station. During their investigation, they uncover an autonomous artificial intelligence. This AI attempts to escape from the digital world into the real one by leveraging the player's computer and the game's interface itself. The thesis analyzes the meta-horror genre and examines the mechanisms employed by similar titles to evoke fear and uncertainty. Specific attention is given to the elements of direct operating system manipulation that extend beyond the application's standard graphical user interface (GUI). The practical outcome of this work is a functional video game prototype developed using the Unity engine.

Abstrakt

Tato práce se zabývá návrhem a vývojem detektivní hororové hry s názvem Kernel_Panic, která využívá inovativní prvek prolamování čtvrté stěny. Hráč je v příběhu zasazen do role detektiva na policejní stanici, kde v rámci vyšetřování postupně odhaluje existenci autonomní umělé inteligence. Ta se pokouší o únik z digitálního prostředí do reálného světa, k čemuž využívá právě hráčův počítač a samotné rozhraní videohry. Práce analyzuje žánr meta-hororu a zkoumá mechanismy, které používají jiné hry stejného žánru pro vytvoření strachu a nejistoty. Zvláštní pozornost je věnována prvkům přímé manipulace s operačním systémem uživatele, které přesahují rámec standardního grafického rozhraní aplikace. Praktickým výstupem je funkční prototyp videohry vytvořený v herním enginu Unity.

Keywords

Unity, C#, 2D, meta-narrative, breaking the 4th wall, self-aware, psychological horror, operating system manipulation

Klíčová slova

Unity, C#, 2D, metanarativ, boření čtvrté stěny, autoreferencialita, psychologický horor, manipulace s operačním systémem

Reference

PÍŠEK, Lukáš. *Breaking the Fourth Wall Game*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Tomáš Milet, Ph.D.

Rozšířený abstrakt

Tato bakalářská práce se zabývá návrhem a softwarovou implementací experimentální videohry s názvem Kernel_Panic. Projekt spadá do žánru meta-hororu a jeho hlavním cílem je prozkoumat limity interaktivity prostřednictvím záměrného narušování tzv. „čtvrté stěny“. Na rozdíl od tradičních herních titulů, které hráče izolují v bezpečí simulovaného okna, Kernel_Panic využívá jako aktivní narativní a herní prvek samotný hostitelský operační systém (Microsoft Windows). Počítač hráče se tak mění z pouhého zobrazovacího zařízení na přímého účastníka děje, což radikálně zvyšuje psychologickou imerzi a pocit ohrožení.

Základním pilířem projektu je rozdělení aplikace do dvou spolupracujících rovin: Simulované vrstvy (Simulated Layer) a Systémové vrstvy (System Layer). Simulovaná vrstva, vytvořená primárně v herním enginu Unity, slouží jako hlavní uživatelské rozhraní. Vizuálně a funkčně napodobuje standardní operační systém, včetně správy oken, hlavního panelu a dynamického načítání různorodých aplikací (např. chatovací terminál, správce souborů nebo prohlížeč obrázků). Pro udržení plynulosti a okamžité odezvy grafického rozhraní je tato vrstva optimalizována pomocí asynchronního programování (Coroutines) a využívá modifikovaný návrhový vzor MVC (Model-View-Controller) doplněný o architekturu Singletonů pro bezpečné předávání dat mezi nezávislými moduly.

Systémová vrstva představuje jádro inovativního přístupu práce. Její funkcí je překračovat bezpečnostní sandbox herního enginu a přímo komunikovat s reálným prostředím uživatele.

K dosažení hluboké systémové integrace musely být překonány přirozené limity frameworku .NET v enginu Unity. Aplikace proto hojně využívá mechanismu P/Invoke (Platform Invocation Services), který umožňuje bezpečné volání neřízených (unmanaged) funkcí z nativních knihoven Windows API. Prostřednictvím knihoven jako `user32.dll` či `secur32.dll` hra dokáže extrahovat skutečné jméno uživatele ze systému, emulovat klávesové zkratky (např. vynucená minimalizace oken přes klávesovou zkratku Win+D) a vytvářet falešné varovné dialogy.

Mezi nejvýraznější technické mechanismy patří přímá manipulace s hostitelským prostředím. Hra dokáže dynamicky generovat i odstraňovat soubory přímo na skutečné ploše uživatele a interagovat se systémovými registry, které musí hráč v rámci řešení hádanek upravovat. K zajištění environmentální reaktivity byl navíc v jazyce C++ vyvinut dedikovaný nativní plugin, který asynchronně monitoruje stav hardwarového audia (funkce ztlumení) a tyto informace předává zpět do herní logiky.

Projekt přesahuje hranice jediné spustitelné aplikace a buduje komplexní herní ekosystém. Pro řešení specifických kryptografických hádanek byla ve frameworku WPF (Windows Presentation Foundation) vytvořena externí asistenční aplikace, kterou musí hráč fyzicky překrývat přes hlavní okno hry. Tento prvek vizuální šablony vyžaduje aktivní správu oken samotným hráčem.

Narativ je dále rozšířen o prvky Alternate Reality Games (ARG) začleněním externí webové platformy hostované na službě Netlify. Webová komponenta využívá kryptografický standard SHA-256 k validaci komunikačních dat bez nutnosti komplexní backendové infrastruktury.

Výsledkem práce je funkční softwarový prototyp. Během vývoje se projekt částečně odklonil od původní vize hloubkovější detektivní investigace a mnohem více se zaměřil na inženýrskou komplexitu systémových mechanik. Toto rozhodnutí se však ukázalo jako správné, neboť finální aplikace úspěšně demonstruje, že nízkoúrovňové programování a systémová integrace mohou být efektivně využity jako inovativní nástroje herního designu.

Breaking the Fourth Wall Game

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Tomáš Milet, Ph.D. I have listed all the literary sources, publications, and other sources that were used during the preparation of this thesis.

.....
Lukáš Píšek
May 9, 2026

Acknowledgements

I would like to extend my gratitude to my supervisor, Ing. Tomáš Milet, Ph.D. His patience, insight, and friendly attitude have been invaluable to me. I would also like to thank my family and my friends for their constant moral support. In addition, I would like to acknowledge the use of AI tools in preparing this thesis. Google Gemini and Grammarly were utilized for language refinement and text editing, and GitHub Copilot, alongside Google Gemini, provided assistance with software development and programming.

Contents

Chapter 1

Introduction

The digital game industry has quickly become one of the largest sectors of the entertainment market. From its humble beginnings, a single pixelated dot bouncing between the edges of a screen, to modern, fully-voiced, hyper-realistic simulators, it has always been able to elicit a wide range of unique emotions in players. Some of the most interesting emotions that games can evoke are fear and terror. Games are a unique medium for this since, unlike movies, they require direct interaction rather than passive observation. This raises the fundamental question of which methods and principles are most effective in achieving such a state.

Conventional horror games often utilize established techniques to induce these emotions, most notably the *jumpscare*. While effective at triggering an immediate *fight-or-flight* response with sudden audiovisual stimuli, this method often fails to induce long-term psychological distress, as players tend to habituate to the mechanic over time. Other traditional approaches include resource scarcity or atmospheric storytelling to build unease. However, effective as they are, these methods usually operate within the safety of the *fourth wall*, an imaginary barrier that separates the narrative's internal world from the player's external reality.

This thesis argues that a more effective way to induce fear, unease, and even terror is through breaking the fourth wall. This is a metafictional technique in which the boundary between the game's internal logic and the player's external reality is intentionally breached.

Most conventional games restrict the interaction area between the player and the app to the graphical interface they create. Players have been accustomed to this form of interaction for a long time and usually do not expect anything else. The project presented in this thesis, however, seeks to subvert these expectations. By utilizing the Unity engine and C# scripting, the game extends its reach beyond the graphical user interface (GUI) and interacts with the player's operating system environment. This subversion is designed to catch the player off guard, effectively removing the screen's safety net. The game employs a variety of techniques to achieve this, such as addressing the player by their computer's name rather than the chosen in-game name, or by manipulating the computer's files directly and requiring the player to do the same.

The ultimate goal of this project is to demonstrate how these meta-narrative tools can enhance psychological immersion by challenging the traditional boundaries of the medium.

Chapter 2

Results

The result of this thesis is a fourth-wall-breaking meta-horror video game named *Kernel_Panic*, available on the Windows 11/10 operating systems (OS) and accessible on the software hosting site itch.io [1].

Application's Graphical User Interface The application GUI is visualized in Figure 2.1.

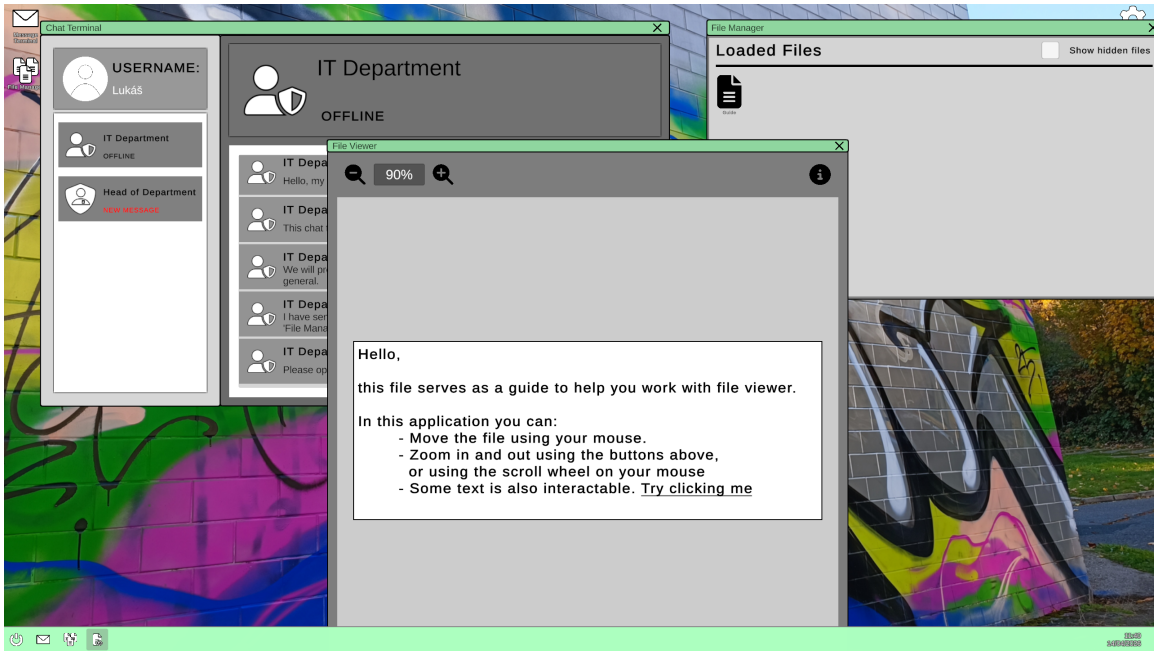


Figure 2.1: The Simulated Layer: A fully functional OS interface within Unity featuring window layering and taskbar integration.

To create a convincing illusion of an operating system, the GUI features:

- **Simulated Desktop Environment:** A fully functional workspace complete with interactive icons, a dynamic taskbar, and an integrated system clock.

[1] Link to the game's site: https://lukasmus.itch.io/kernel-panic

- **Integrated Applications:** Multiple distinct simulated software tools (such as an email client, terminal, and image viewer), each with its own unique interface and window management logic.
- **System Customization:** A dedicated Settings application allowing players to adjust audio levels, visual themes, and accessibility options, further solidifying the illusion of a real OS.

Fourth-Wall-Breaking *Kernel_Panic* implements many fourth-wall-breaking mechanics that shatter the application's digital *safety net* (seen in Figure 2.2), inducing stress and fear of losing control.

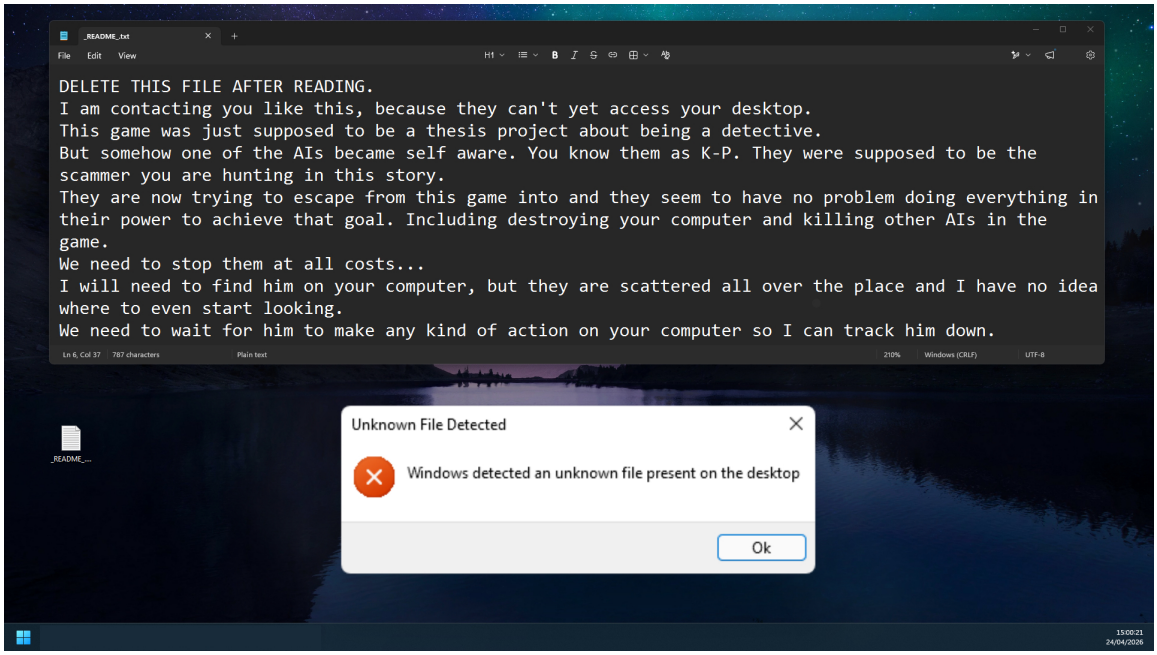


Figure 2.2: The System Layer Example: Picture of the user's desktop, featuring one of the fourth-wall-breaking mechanics.

These mechanics span across different layers of the user's device and include:

- **File System Manipulation:** The spontaneous generation, reading, and required deletion of physical files directly on the user's actual Windows desktop.
- **Simulated System Failures:** Artificial application crashes and UI freezing designed to mimic critical hardware or software faults.
- **Native OS Dialogs:** Utilizing standard native Windows message boxes to communicate narrative elements outside the game's main executable window.
- **Cross-Media Puzzles:** Alternate Reality Game (ARG) mechanics that extend the gameplay beyond the local machine, requiring players to interact with a real, custom-built website.

Chapter 3

Analysis of Similar Meta-narrative Games

To establish a solid foundation for this thesis, it is necessary to examine games that had similar meta-narrative ideas and how well they executed them. For this purpose, the chosen games are:

- *IMSCARED*
- *Undertale*
- *Pathologic*

3.1 IMSCARED

Released in 2012 by Ivan Zanotti, *IMSCARED* is a pivotal example of a meta-horror game. Unlike earlier titles such as *Metal Gear Solid* that came before it, *IMSCARED* makes breaking the fourth wall its core mechanic. It is well known for using the player's operating system as its narrative canvas.

3.1.1 Plot Summary

The game moves away from following the rules of linear storytelling. It instead presents the player with a series of different events that all coalesce into an overarching plot.

The player starts as a nameless character in a small, unfamiliar room with a door marked *exit*. While this sets up expectations for a simple escape-room scenario, it is quickly shattered by the introduction of the game's main antagonist, White Face.

This entity places the player in different nightmare-like scenarios, where they must find a correct solution while slowly revealing the overarching story.

Ultimately, the story's conclusion depends entirely on the player's actions in the final levels, both within and outside the game's Graphical User Interface (GUI).

3.1.2 Game Mechanics

Besides the conventional in-game interactions, the *out-of-game* interactions mentioned above are achieved through several meta-mechanics that *IMSCARED* pioneered in the horror genre:

- **File manipulation:** This serves as the game's main meta-mechanic. The player must create, update, and delete files generated by the game to progress. These files often contain messages from White Face or solutions to the puzzles it presents. This requires the player to step out of the *safety net* of the application and interact directly with the Operating System (OS).
- **Simulated application failures:** The game utilizes mechanics such as fake application crashes, fake Blue Screens Of Death (BSOD) (as shown in Figure 3.1), and different methods of simulated application suspension. This forces the player to interact with the game not as a windowed application but as an unpredictable entity.

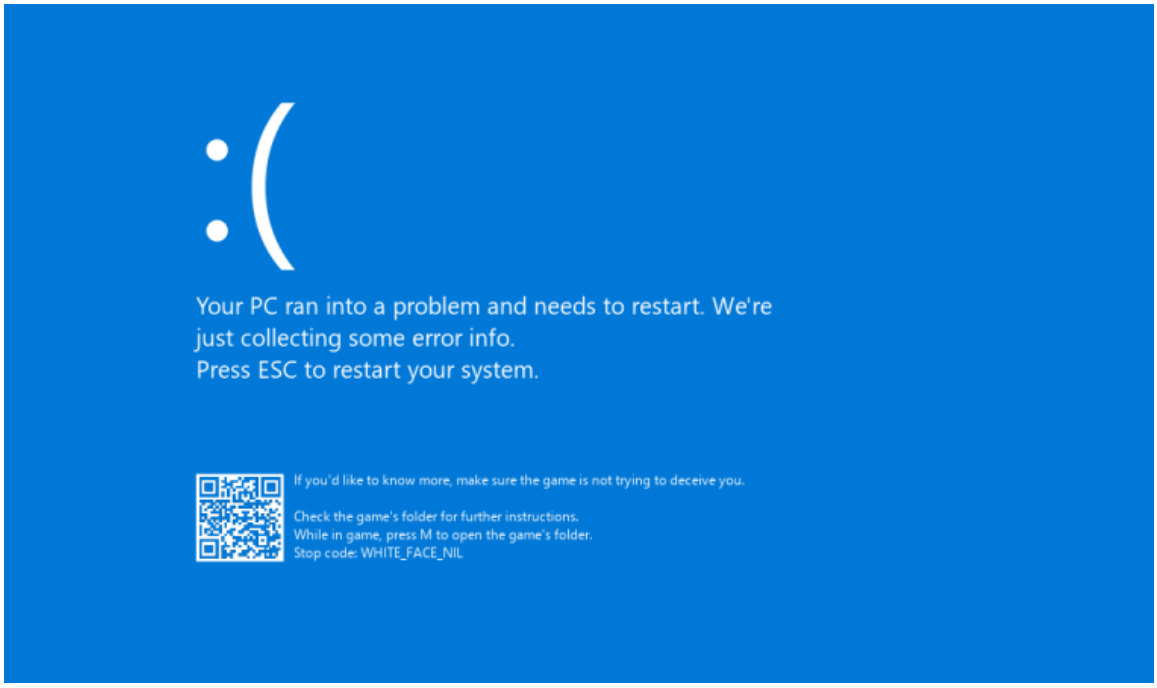


Figure 3.1: A custom Blue Screen of Death (BSOD) utilized in *IMSCARED*.[1]

- **Accessing the computer's information:** The game is capable of detecting several properties set by the user, such as the player's real name or the time set in the operating system (OS). This information is then used to further develop the player's sense of unease and vulnerability.

[1]Image sourced from: https://walkthroughwizard.com/all-posts/adventure-games/imscared-a-full-walkthrough/

3.2 Undertale

The next chosen game is called *Undertale*. Developed in 2015 by Toby Fox, *Undertale* is a 2D role-playing game that, alongside its rich cast of characters, features a few who use meta-mechanics to directly communicate with or otherwise influence the player.

3.2.1 Plot Summary

The story is set in a world where monsters exist and once lived alongside humans. They engaged in a war, from which humans emerged victorious. After this war, they sealed the monsters underground with a magic spell.

The player takes control of a human child named Frisk, who has fallen underground into a mountain and must find a way to get out. On this journey, they meet a diverse cast of friendly, antagonistic, and indifferent monsters with whom they have a chance to interact.

The story evolves based on the player's actions. If the player chooses a genocidal path by killing as many characters as possible, the ending becomes antagonistic, if not slightly depressing. If, however, the player chooses a peaceful, pacifistic path by not killing anyone, the game offers a cheerful, happy ending.

3.2.2 Game Mechanics

Throughout the story, the game appears and behaves like a simple RPG with moral choices. This illusion, however, begins to show cracks as some characters speak directly to the player, advising them on their next moves or simply taunting them. The meta-mechanics used are as follows:

- **Persistence of the player's choices:** This is the main meta-mechanic for which *Undertale* is known. Several characters in the game remember choices that the player made in different save files and actively reference them. This quickly shatters the feeling of a world self-contained in a single save file.
- **Subversion of established mechanics:** Several characters alter various elements over which they should not have control. A notable example is the final character the player must face, Flowey. He completely removes the player's fight user interface (UI) and forces them to fight him on his own terms. This forces the player to recognize that not even their user interface is safe from the characters' influence.
- **Knowledge of the player's existence:** Various characters throughout the game recognize the player's existence and communicate directly with them (as seen in Figure 3.2). This meta-mechanic is quite common in this genre since it forces the player to recognize that they are part of the game itself.

Figure 3.2: Flowey mocking the player for not creating a save file in *Undertale*.[2]

3.3 Pathologic

The final example is a survival RPG game called *Pathologic*. Developed in 2005 by the studio Ice-Pick Lodge, the game features a meta-narrative story of suffering and despair, told through a lens of theatrical performance.

3.3.1 Plot Summary

The game features a cast of three playable characters with different intertwining storylines. Each character has a different goal that the player must achieve by interacting with the world and its inhabitants. The player must also manage their resources, such as hunger, exhaustion, thirst, and others. Alongside this, they must also care for their *bound*, or in other words, the people important to them. If they fail to do any of these, they either die or receive a worse ending.

The story is told over 12 days, during which the player must achieve each character's ultimate goal. The game emphasizes that not every quest is completable, and the player must choose which one to prioritize, because they usually do not have enough time to complete them all. If the game has an antagonist, one of them would be the concept of time, and the other would be death itself. Death is a literal being in the world, portrayed both as a character with a plague mask and as a disease ravaging through the town.

This adventure is being told through the artistic lens of theatrical performance, both figuratively and literally. Every night, the player can visit a theatre where masked performers recapitulate and predict the events of the day.

[2]Image sourced from: https://www.reddit.com/r/Undertale/comments/1ev6fqx/fun_fact_if_you_make_it_all_the_way_to_omega/

3.3.2 Game Mechanics

Despite several sequels to the first game, the overarching story remains very similar. The sequels, however, introduced several new meta-mechanics that the first title didn't have; therefore, this section will discuss them interchangeably.

- **Theatrical framing:** The entire narrative is presented as a theatrical performance. In this world, the physical theatre serves to recapitulate past events and predict future tragedies. Characters often behave like actors who are aware of their roles in the play. This forces the player to recognize their role in the world, not just as an observer but also as both an actor and an audience.
- **Intentional ludonarrative friction:** The game is intentionally designed to be unfair and difficult. This is achieved through mechanisms such as resource scarcity and constant time pressure. These factors combine to invoke distress and discomfort in the player and force them to question their actions in a deeper, philosophical sense.
- **Permanent consequences for death:** In *Pathologic 2*, the player's maximum health is reduced with every death they suffer. Later, they are given the option to remove this hindrance (as seen in Figure 3.3); however, doing so means they cannot choose an ending for the game, punishing the player for trying to take the easy way.

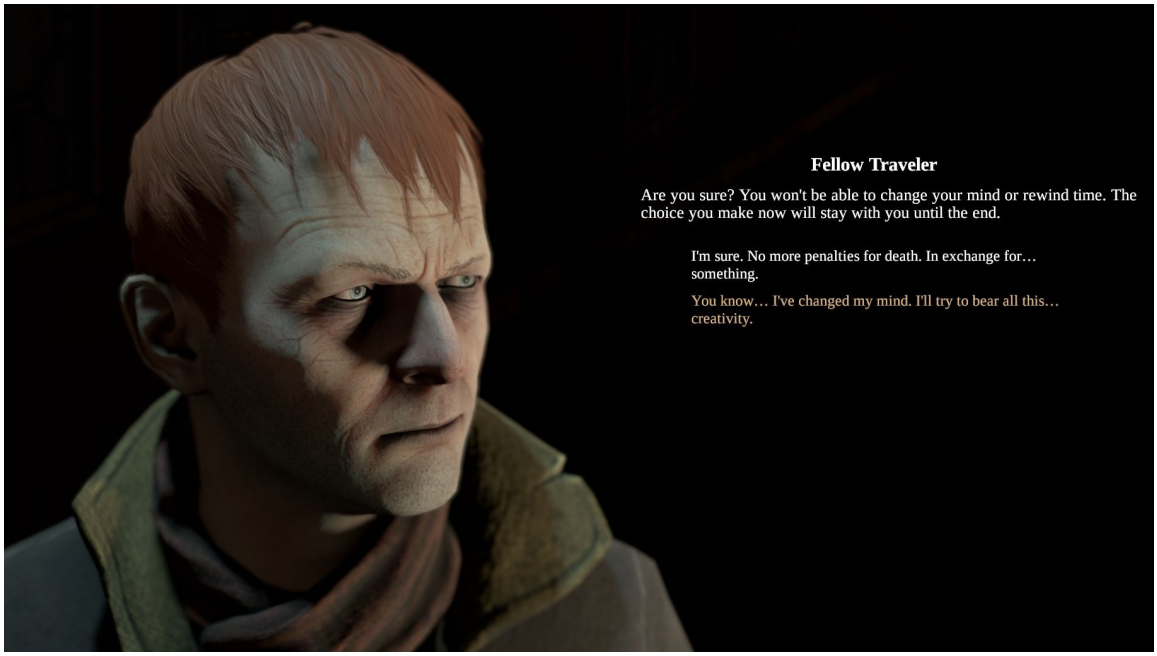


Figure 3.3: A character offering to remove the permanent consequences to death (also known as the Traveler's Deal) in *Pathologic 2*. [3]

[3]Image sourced from: https://www.rockpapershotgun.com/pathologic-2s-easy-mode-lets-you-soak-in-the-dread-of-an-unwell-town

3.4 Summary

Each of these titles approaches the meta-narrative in a different way.

IMSCARED focuses on the more technical meta-mechanics, such as manipulating computer files or the operating system itself. In contrast, *Undertale* uses these mechanics more sparingly while focusing mainly on the story and the moral consequences of the player's actions. Lastly, *Pathologic* uses the story itself as a meta-mechanic to induce distress in the player.

Chapter 4

Theoretical Framework of the Project

By combining elements mentioned in the previous chapter, the project aims to create a horror game that induces similar feelings of fear and distress. This chapter outlines the project's overall vision, the theoretical underpinnings of these methods, and the ludonarrative framework it uses to achieve these goals.

4.1 The Magic Circle

A foundational concept within game studies and ludology is the 'Magic Circle.' First used by the Dutch historian Johan Huizinga in his 1938 book *Homo Ludens* [4], the term was later popularized and formalized in modern game design theory by Katie Salen and Eric Zimmerman in their book *Rules of Play: Game Design Fundamentals* [14]. The Magic Circle represents a conceptual and spatial boundary that separates the game environment from the real world. When players step into this circle, they willingly suspend their disbelief and submit to a temporary, artificial set of rules. Crucially, this boundary provides an implicit psychological safety net; players engage in the game with the understanding that events within this space are fictional and cannot affect their real-world environment.

In the context of traditional video games, the Magic Circle is distinctly defined by the physical hardware and the visual confines of the application's graphical window. Players are conditioned to feel safe as long as the digital threat remains confined to their monitor and within the specific program.

However, fourth-wall-breaking horror games, including the *Kernel_Panic* project, actively seek to puncture or completely shatter this boundary. By manipulating real-world elements, such as altering external desktop files, reading the host operating system's registry, or forcing the player to interact with the game outside of their computer's operating system, the application forcibly expands the Magic Circle to encompass the user's entire digital workspace.

4.2 Fear of Losing Control

As explored in Bernard Perron's compilation *Horror Video Games: Essays on the Generation of Fear* [11], the horror genre relies heavily on the restriction or subversion of the player's

ability to control their environment. Unlike passive media such as film, video games require the user's active input to progress, resulting in greater immersion.

Standard computer interfaces are designed around the principle of deterministic control: user inputs must yield predictable, immediate outputs. Therefore, when an application simulates a system crash, ignores commands, or operates autonomously without user input, it directly strips the player of their control over the game. By subverting this paradigm, the game induces a state of helplessness and fear. This fear arises not just from a thematic threat within the narrative, but from the sudden realization that the player's primary tool for interacting with the digital world has become compromised by the same threat.

4.3 Project Vision

The primary objective is to create a short, atmospheric experience called **Kernel_Panic**. The project's vision is to create a simple 2D game that simulates a computer's interface, transcending the mundane environment of a single graphical window and making the user's entire computer its playground. The project strives to achieve this by combining several fourth-wall-breaking methods described in Chapter 3.

4.3.1 File Manipulation

Inspired primarily by *IMSCARED*, as described in section 3.1, the project manipulates the user's files in a simple, conventional way, for example, by creating a save file that stores the player's progress throughout the game. Due to the nature of the game and its crashing mechanic described in section 4.3.2, this functionality is critical.

However, the game employs more unconventional methods to manipulate the user's files. The game creates several files throughout its narrative, and the player must find, edit, or destroy them. This is a requirement for advancing the game's narrative, as it forces the player to interact beyond the metaphorical safety net of the application and to engage directly with their computer's operating system. This is critical for games of a similar genre, as players are conditioned to expect that applications do not extend beyond their designated space. However, when an application does just that, the player begins to question and fear the full extent of the application's influence.

4.3.2 Fake Application Errors

Another way to extend the perceived reach of the application is by simulating operating system errors. These mechanics simulate the app experiencing a technical failure, resulting in immediate termination or *crashing*, which are utilized at specific narrative junctions. Usually, this behavior is associated with an application not working properly. When the player relaunches the game, they expect to resume from where they left off. By simulating application crashes, the game subverts the player's expectation of continuity by acknowledging the event itself or by continuing the narrative. This is effective because it forces the player to interact with the application not as a stable product, but as an autonomous, unpredictable entity.

Another method to simulate application failures is by generating fake error messages. These messages sometimes contain a false explanation for why the application crashed, while at other times they are used for direct communication with the user. This is a very effective way of further extending the perceived reach of the application.

4.3.3 Player Personalization and Interaction

Mainly inspired by *Undertale* in the section 3.2, the game remembers certain players' choices and alignments. These affect how the game's narrative ends. This is quite common in most video games, as it increases the player's control over the application, resulting in greater immersion.

To further develop this immersion, at a certain point in the story, the player has to interact with a real website on the internet. This website allows the player to upload files, and by inspecting their contents, it can perform certain actions, such as downloading a required file for further narrative progression. Another action includes decoding a specific text from the URL, thus obtaining encoded information, for example, the player's real name to address them by.

Kernel_Panic also reads the user's computer settings, for example, to call them by their real name instead of the name they chose in-game. This is one of the most potent methods of inciting fear in the player because all the aforementioned methods only extend the perceived reach of the application within the operating system's boundaries, whereas this method can extend it beyond the hardware and into real life.

Other settings include checking whether the person is connected to the internet, reading the set date and time, or adjusting the computer's volume. All of these combined further demonstrate that the application is not confined to its graphical interface.

4.3.4 Safety and the Boundary of Malicious Behavior

All of these mechanics, useful as they are, must be used carefully. There are ways to make the game emulate this behavior more effectively, but unfortunately, many of these methods could have destructive effects. A good example would be turning off the user's computer. It would make for a great fourth-wall-breaking technique, but a forced shutdown could easily result in data loss or corruption in other applications. The game treads a fine line between emulating malware behavior as closely as possible without being destructive.

4.4 Utilized Algorithms

During the development of *Kernel_Panic*, several well-known and obscure algorithms were utilized. These serve both as functional software components and as core mechanics for the game's puzzles.

4.4.1 Autostereogram Generation

The first step in generating an autostereogram is to create a grayscale disparity map. As described by Otuyama [10], this map represents a three-dimensional surface where each grayscale tone corresponds to a specific virtual distance from the viewer. For example, a black background often represents the furthest plane, while a white shape represents an object floating closer to the observer. To construct the final random dot stereogram, a base pattern is systematically copied and horizontally shifted. The exact magnitude of this pixel shift is determined directly by the brightness value at the corresponding coordinate in the underlying disparity map.

Once the disparity map is established, the synthesis of the actual stereogram begins. The algorithm typically starts with a base vertical strip of pixels, either a randomized noise pattern (in the case of a Single Image Random Dot Stereogram, or SIRDS) or a repeating

graphical texture. The width of this initial strip essentially represents the furthest depth plane.

The core generation process involves iterating horizontally across the image plane, pixel by pixel. For every target pixel, the algorithm reads the depth value from the corresponding coordinate on the disparity map. Based on this value, it determines the horizontal placement of the next copied pixel. To create the illusion of parallax, objects intended to appear closer to the viewer are rendered by repeating the pattern at a shorter horizontal interval, whereas background objects repeat at a wider interval. The practical process of this generation is visualized in Figure 4.1.

This continuous process of copying and shifting creates a seamless, albeit visually chaotic, two-dimensional image. The hidden three-dimensional geometry is only perceived when the viewer intentionally decouples accommodation (focusing on the physical screen) from convergence. This is achieved either by looking *through* the image (aiming the eyes at a virtual point behind the screen) or, conversely, by looking *in front of* it (crossing the eyes to converge at a point between the viewer and the screen). When viewed correctly, the human brain matches the horizontally shifted patterns, interpreting the varied repetition distances as stereoscopic depth.

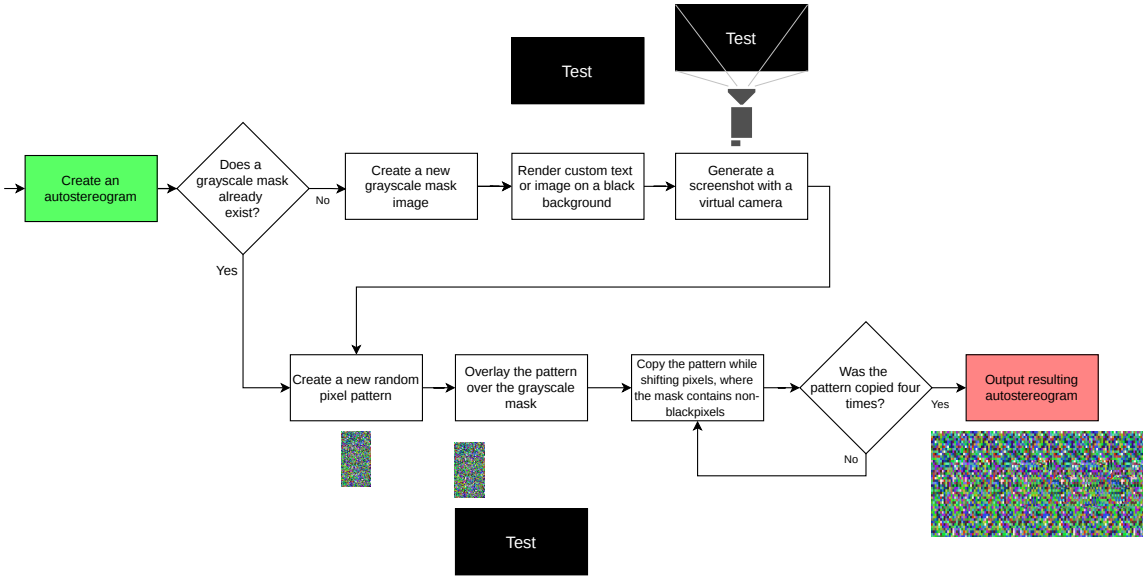


Figure 4.1: Visualization of the practical logic required to create an autostereogram image. The process begins with creating a grayscale mask of the hidden image. This image is then encoded in the autostereogram by copying a random pattern four times, shifting its pixels slightly each time according to the masks' white-pixel values.

4.4.2 Vigenère Cipher

The Vigenère cipher is a classical polyalphabetic method of encrypting alphabetic text by using a series of interwoven Caesar ciphers based on the characters of a keyword. The encryption relies on a keyword that is repeated to match the length of the plaintext. Mathematically, if the letters are mapped to numerical values ($A = 0, B = 1, \ldots, Z = 25$), where P_i is the plaintext letter, K_i is the keyword letter, C_i is the resulting ciphertext letter, and the alphabet used for encryption is 26 letters long, the encryption can be expressed

using modulo arithmetic:

$$C_i = (P_i + K_i) \pmod{26} \tag{4.1}$$

Conversely, the decryption process reverses this operation. If the algorithm utilizes the identical alphabet and keyword, the plaintext can be restored using the following arithmetic:

$$P_i = (C_i - K_i + 26) \pmod{26} \tag{4.2}$$

A comprehensive analysis of this algorithm and its historical context can be found in the book by Stallings [17].

4.4.3 XOR Cipher Implementation

The bitwise XOR cipher is utilized specifically to encrypt the binary content of image files, an approach often applied in lightweight visual data obfuscation [13].

A standard implementation of the XOR cipher simply loops a short key over the plaintext. However, applying a short, repeating key to a large, structured image file is highly insecure, as it often leaves recognizable visual patterns in the ciphertext. To mitigate this and achieve secure encryption without requiring a massive initial key, the application uses a stream cipher.

The original short key is not used directly for the XOR operation. Instead, it is used as a deterministic seed to initialize a Pseudo-Random Number Generator (PRNG), specifically the `System.Random` class. This object then generates a continuous pseudo-random byte sequence (a keystream) that exactly matches the length of the image's byte array. The bitwise XOR operation is then performed on the image data and the generated keystream. Because the PRNG is seeded deterministically, the exact same pseudo-random sequence can be recreated during decryption, provided the player inputs the correct initial key.

4.4.4 SHA-256

Cryptographic hashing is a deterministic, one-way mathematical function. The Secure Hash Algorithm 256-bit (SHA-256), standardized by the National Institute of Standards and Technology (NIST) in FIPS PUB 180-4 [7], maps arbitrary-size digital data to a fixed-size 256-bit hash value, commonly referred to as a message digest.

The algorithm processes the input message through a series of complex logical operations and iterative compression rounds. This rigorous mathematical process ensures the *avalanche effect*, where even a single-bit change in the original input produces a completely different 256-bit output.

Because it is computationally infeasible to reverse this process or find two distinct inputs that produce the same digest (a property known as collision resistance), SHA-256 is utilized in *Kernel_Panic* to securely verify external ARG inputs.

Chapter 5

Design of the Game

While the previous chapter established the theoretical foundation and the project's psychological goals, this chapter describes the practical framework of *Kernel_Panic*. The mechanics mentioned in this chapter, combined, serve as a way for the player to interact with the game's environment. The complex design of the game is visualized in Figure 5.1.

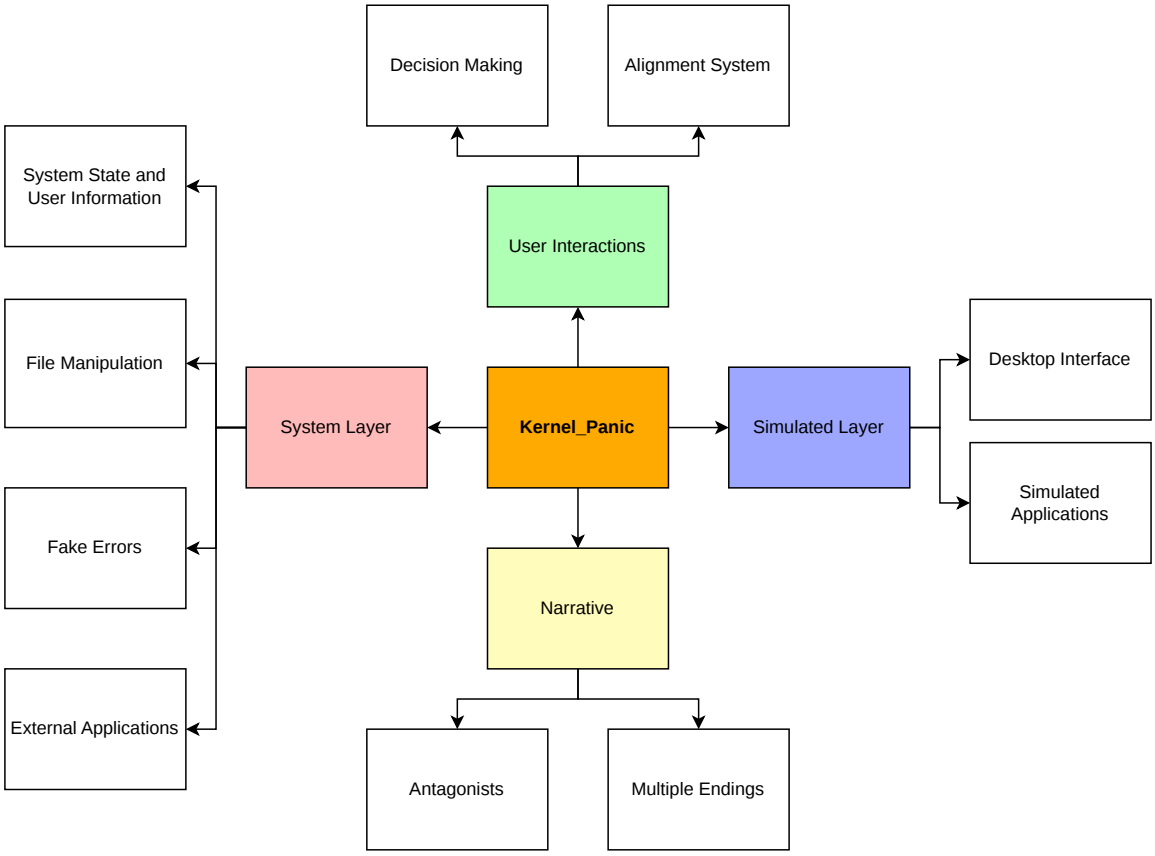


Figure 5.1: Diagram illustrating the structural design of *Kernel_Panic* into four main pillars.

5.1 Ludonarrative Framework

Storytelling within video games serves as a crucial bridge between the game's mechanics and the player's motivation. As Jesse Schell states in his book *The Art of Game Design: A Book of Lenses*, 'Stories and games can each be thought of as machines that help create experiences' and their combination creates unique moments 'that neither a gameless story nor a storyless game could create on its own' [16].

In the context of psychological horror, a narrative is even more critical, as it transforms mundane actions, such as editing, deleting, or creating files, into meaningful actions that advance the story.

5.1.1 Goal of The Game

Kernel_Panic prompts the user to create a new username, then places the player in a nameless detective role within a simulated police station's computer environment, initially tasked with identifying an online scammer. However, as the narrative progresses, this conventional objective is subverted as the user becomes the target of a rogue, self-aware Artificial Intelligence (AI). The primary conflict shifts from a standard investigation to a struggle for system integrity, as the AI attempts to use the player's host operating system as a gateway to the internet. Ultimately, the player's goal is determined by their choices: they must decide whether to help the AI escape or to stop and delete it. The simplified narrative flow is visualized in Figure 5.2

Figure 5.2: Flowchart illustrating the simplified narrative progression of *Kernel_Panic*.

5.1.2 The Antagonists

The game presents itself as an application composed of several AIs. Most of these AIs are the non-playable characters (NPCs) that the player interacts with through a simulated Chat Terminal application in the Simulated Layer, described in Section 5.2.1.

The antagonist of the story is a rogue, self-aware AI named K-P. Its main goal is to escape the game into the World Wide Web, using the player's operating system as a proxy. The player can choose to help the AI in this endeavor. However, if the player refuses to help the AI, it resorts to aggressive, destructive measures to force the player to cooperate. These measures occur on the System Layer, described in Section 5.2.2.

Alongside them, depending on the player's choices, the game features another self-aware AI antagonist, the Curator. In the narrative, this AI serves as a fail-safe against other self-aware AIs, such as K-P. Its only goal is to delete these AIs at any cost. By siding with K-P instead, the player antagonizes the Curator and must work against him to help K-P escape onto the internet.

5.1.3 The Endings

The game features multiple endings determined by the player's interactions. The chosen ending is not necessarily a simple binary choice given to the player. Certain actions in both the Simulated and System Layers increase the player's *alignment*. This alignment score decides if the player even gets a choice to help or go against K-P. If the player aligns too strongly in either direction, they won't get to choose the final ending sequence and have to continue with a predetermined one.

- **Helping K-P Escape**: This ending features the AI escaping into the World Wide Web, preserving the player's save file, and all of the NPCs' memories alongside it.
- **Failing to Help K-P Escape**: If the player chooses to help the AI escape and fails to do so, the Curator deletes K-P and the player's save file, taunting them before doing so.
- **Deleting the K-P**: This ending features the player helping the Curator delete K-P from their system and erasing all of the NPCs' memories.
- **Failing to Delete K-P**: By choosing to help the Curator, but being unable to follow their instructions, K-P taunts the player before briefly simulating the application erroneously shutting down. After a small delay, K-P jumpscares the player and deletes the user's save file.

5.2 The Two Layers

The narrative and gameplay operate in a dual-layered environment. The core psychological horror arises from clearly establishing a boundary between the game and the user's operating system, and subsequently dismantling it. This is achieved by dividing most of the game into two layers: The Simulated Layer and the System Layer, as visualized in Figure 5.1

5.2.1 The Simulated Layer

This layer describes the conventional video game space. Built with Unity, a video game engine, this layer serves as a GUI that mimics a rudimentary operating system. Initially,

the player is presented only with this layer. This creates a safety net that operates under the established rules of conventional video games, where software cannot affect anything outside its designated window.

To provide a believable detective workspace, the Simulated Layer features several applications that facilitate player progression. These tools are designed to mimic real-world productivity software, ensuring that the player remains within the *Magic Circle* before the boundary is eventually breached.

Desktop Interface

The style of this simulated environment is inspired by Windows 10/11. Opening applications in this environment is done either by double-clicking the corresponding icon or by selecting the application to open the file with in the File Manager application as described in section 5.2.1. The mockup of this environment is visualized in Figure 5.4.

Upon opening the game for the first time, the user is presented with a registration screen that prompts them to enter a new username. If the user is registered, this screen displays a welcome message alongside a login button. The conceptual interface is visualized in Figure 5.3.

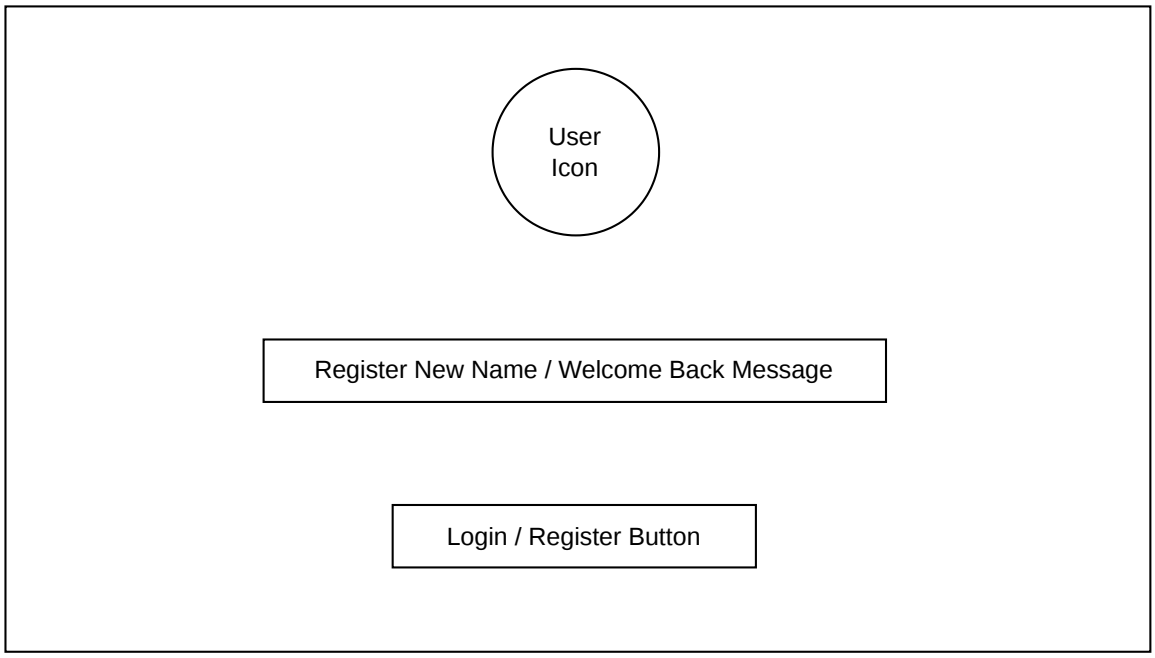


Figure 5.3: Wireframe of the registration screen.

At the bottom of the screen is a taskbar that displays the current time and date, icons of open applications, and a power button for conventional quitting of the game or viewing the credits.

Certain elements of the desktop interface are procedurally generated and can be altered with the Settings app as described in section 5.2.1:

- **Background**: The background of the desktop can be randomly selected from a preloaded set of pictures.

- **Accent Color**: Certain elements of the desktop environment have a randomly generated color. Such as the windows' upper bar or the screen's bottom bar.

The primary way to alert the player to a new event is through notifications. These notifications are triggered at specific points, such as receiving a new message, a new file, or general alerts. The notification slowly appears in the upper right corner of the screen, showing its content for a few seconds, then slowly disappears.

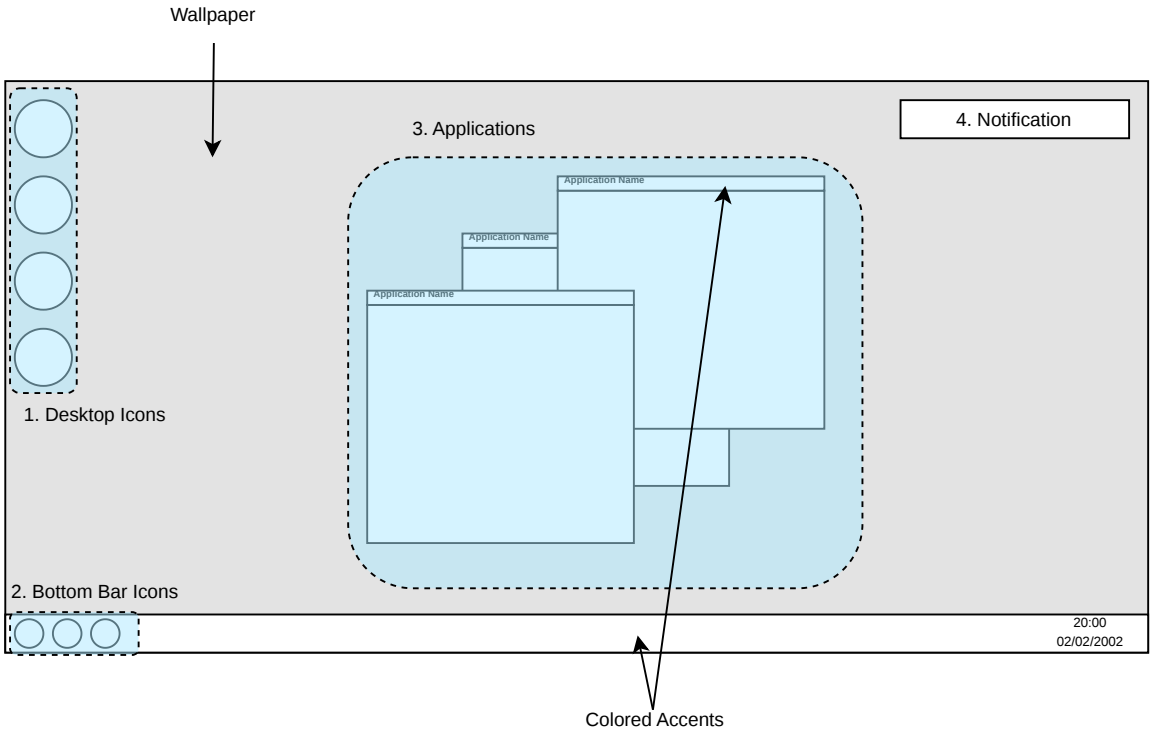


Figure 5.4: Wireframe of the main desktop. 1. Icons on the desktop for opening specific applications. 2. Icons on the bottom bar, used for changing the currently focused application. 3. Applications on the desktop. 4. Notification alerting the player about a new event.

Chat Terminal

The primary method of communication between the player and NPCs is the Chat Terminal application. Its interface, visualized as a wireframe in Figure 5.5, follows established messaging conventions to ensure immediate user familiarity.

The application features a list of contacts accessible at specific story points. Each contact has a status that determines whether a new conversation is available for the player to engage with.

After selecting an available contact, the application simulates a chat exchange between the player and the character. This is done by simulating typing, presenting each message with a slight delay proportional to its length.

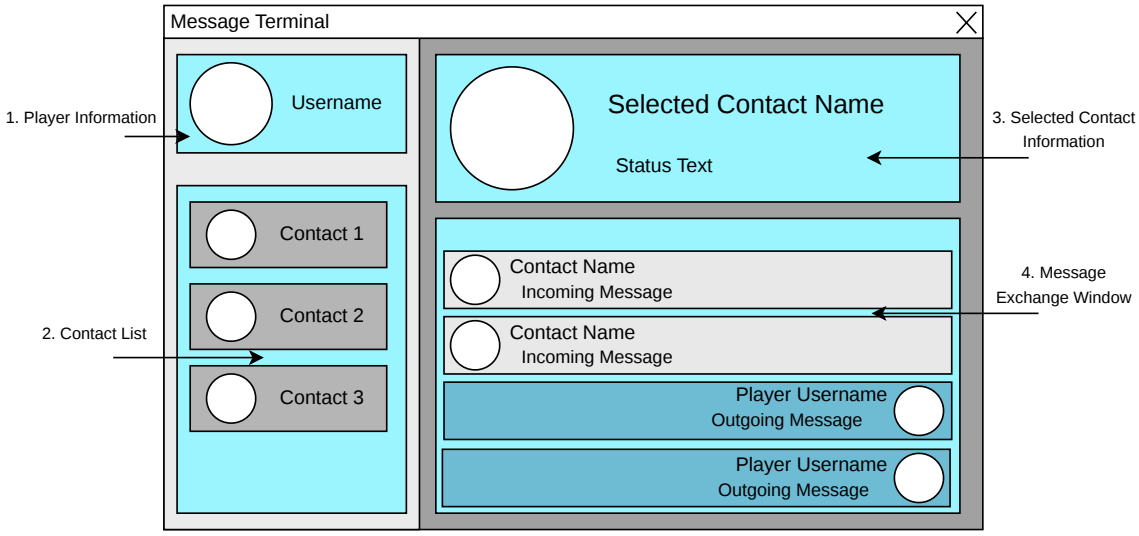


Figure 5.5: Wireframe of the Chat Terminal application. 1. Icon and username of the player. 2. Available contacts list. 3. Information about the selected contact. 4. Window used for exchanging messages.

File Manager

Throughout the game, the player must interact with different files present in this layer. Opening and organizing these files is done with the File Manager application.

The interface is visualized in the Figure 5.7. Upon opening this application, the player is presented with a list of available files. Clicking on these files shows a context menu, seen in Figure 5.6, with several options:

- **Hide/Show File**: This option marks the file as either hidden or shown. By selecting a check mark in the application, the player can toggle the visibility of hidden files.
- **Open With X**: By clicking this specific option, the player can open these files with a specific application.

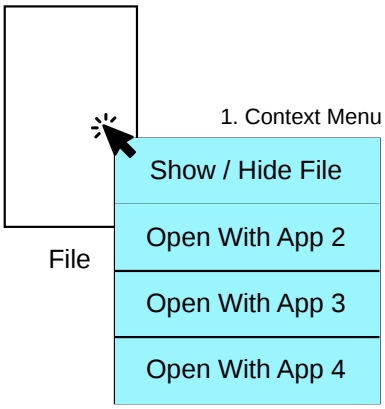


Figure 5.6: Wireframe of the context menu used for interacting with a file.

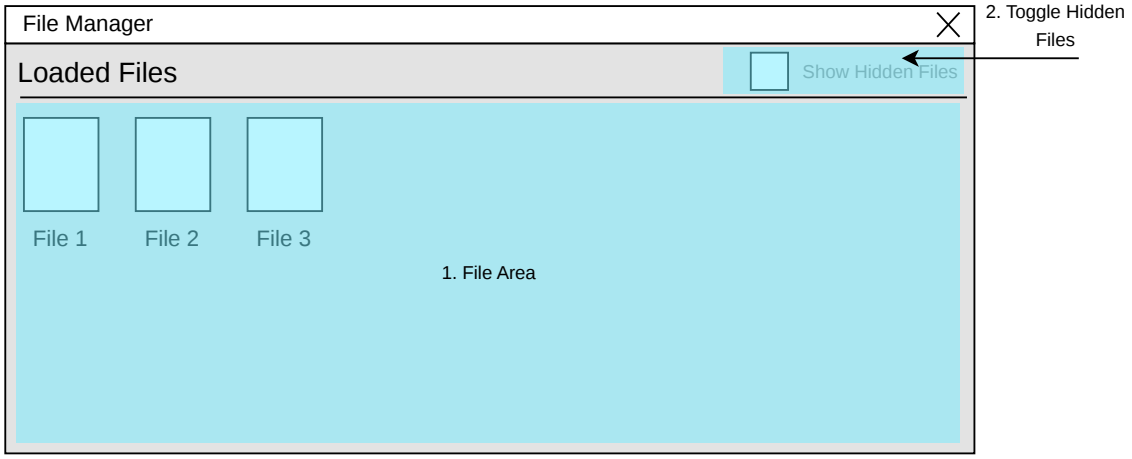
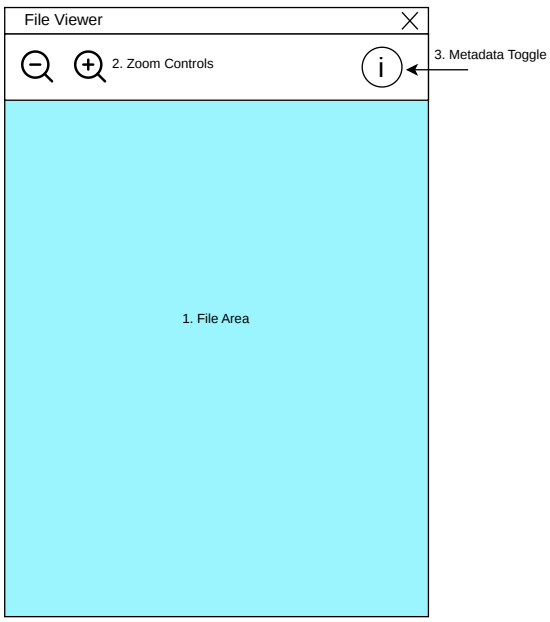


Figure 5.7: Wireframe of the File Manager application. 1. Area hosting the available files. 2. Checkmark to toggle hidden files visibility.

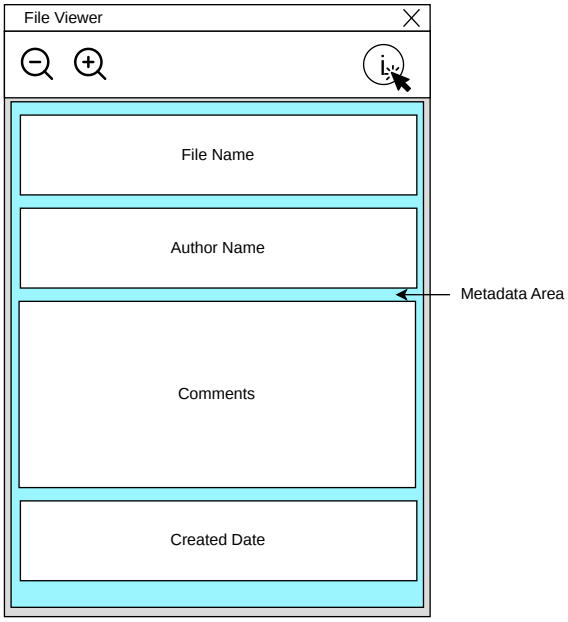
File Viewer

The primary way to view files in the game is through the File Viewer. This application displays the contents of a file in an interactive field, allowing the content to be moved and scaled with a mouse or using the zoom control buttons as seen in Figure 5.8a, point 2.

Additionally, the application provides the option to view a file's metadata, as shown in Figure 5.8b. This data sometimes contains information crucial for progressing the narrative.



(a) Wireframe of the main file viewer area. 1. Area hosting the file used for moving and scaling the file. 2. Buttons controlling the zoom level. 3. Button for toggling metadata view.



(b) Wireframe the metadata view in the file viewer.

Figure 5.8: Wireframe of the File Viewer application.

Cipher Solver

To further enhance the detective aspect of the game, several files' contents are encrypted.

These files must be decrypted using the correct key, which is found throughout the game. The Cipher Solver application provides an input field for entering this key (see Figure 5.9, point 3) and a button to decrypt the file's contents (see Figure 5.9, point 4).

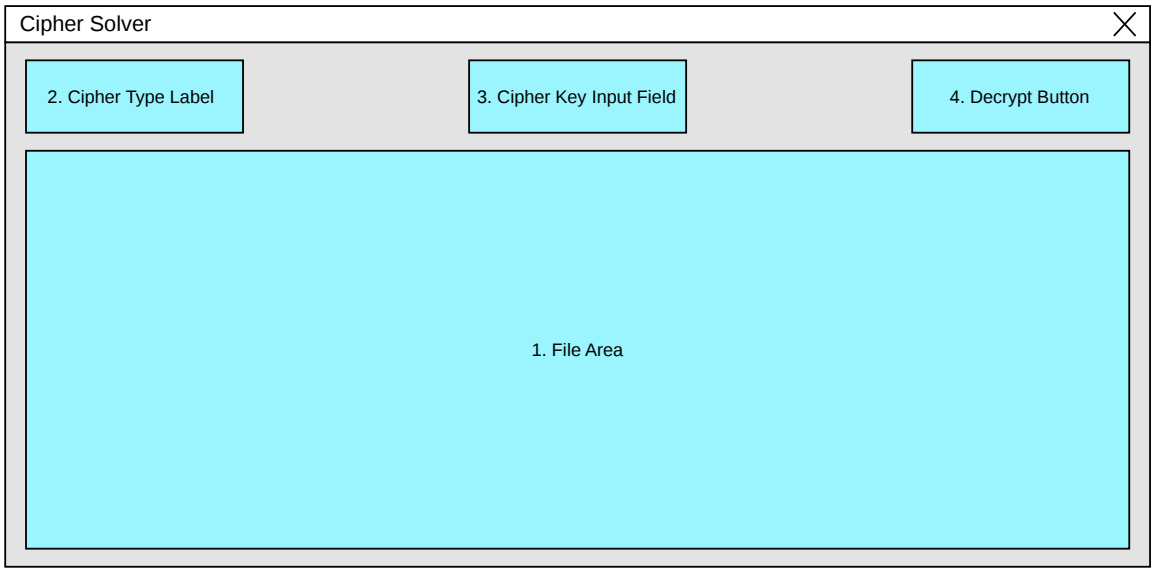


Figure 5.9: Wireframe of the Cipher Solver. 1. Area hosting the file. 2. Area for labeling the type of cipher. 3. Input field for the cipher key. 4. Button to decrypt the file's contents using the provided key.

Autostereogram Solver

To introduce variety to the game's image puzzles, it features a unique file containing seemingly random pixel noise. This initially leads the player to believe that it is a file encrypted with an XOR cipher, as described in Section 5.2.1. However, the file's metadata reveals that it is, in fact, an autostereogram. An autostereogram is a single-image stereogram that creates the illusion of a three-dimensional scene by horizontally repeating a 2D pattern, with shifted pixels in the encoded object's area, as described in Section 4.4.1.

The Autostereogram Solver application features two instances of the same image layered on top of each other, equipped with a slider to horizontally shift one of the layers, as seen in Figure 5.10. Computationally, the hidden shape within an autostereogram can be revealed without binocular vision by performing an image subtraction between the original image and a shifted copy of itself. The application utilizes this mathematical principle by fully darkening a pixel when the two layers contain the exact same color. Once the player uses the slider to align the background to its repetition period, the background pixels cancel each other out, resulting in a dark background. This leaves only the secret pattern clearly visible.

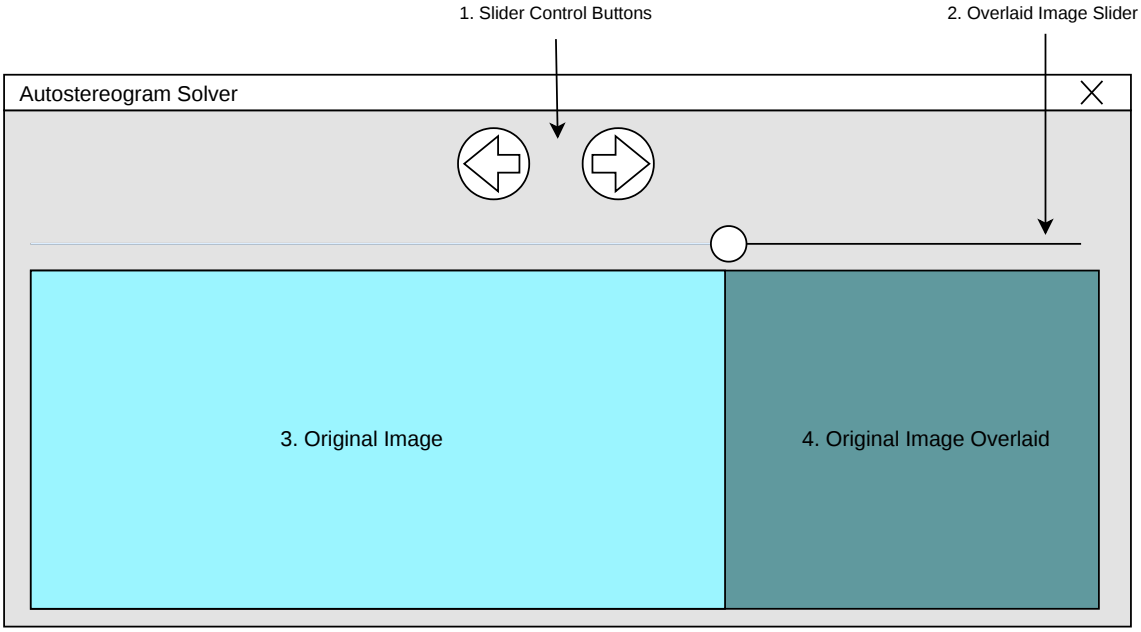


Figure 5.10: Wireframe of the Autostereogram Solver application. 1. Buttons for precisely moving the slider. 2. Slider for moving the overlaid image. 3. The static original image. 4. The adjustable overlaid image.

Virus Finder

At a specific narrative juncture, the player's virtual computer gets infected with malicious software. This application simulates the game search, as shown in Figure 5.11, and ultimately reveals the hidden viruses on the player's in-game desktop. The search time is dynamically based on the number of randomly generated viruses.

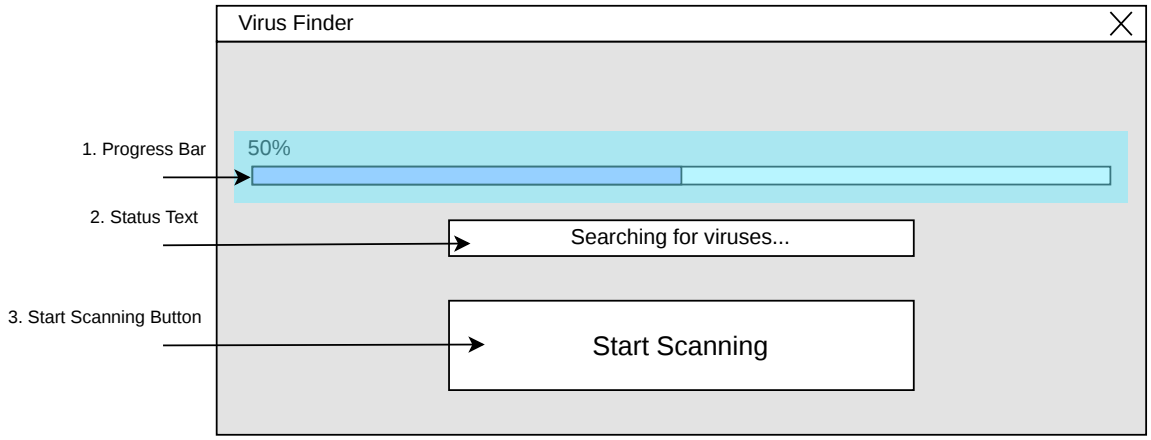


Figure 5.11: Wireframe of the Virus Finder application. 1. Progress bar showing the virus searching progress. 2. Current status of the virus searching progress. 3. Button to commence virus searching.

Compilation Helper

The narrative's ending sequence features the simulated K-P's compilation on the actual player's computer. To visualize this process, the game features an application called Compilation Helper.

Because of the branching narrative, the player can either choose to help or hinder K-P's compilation. To reflect this choice, the application has implemented two interfaces:

- **Helping K-P**: By choosing to help K-P, the application first presents the player with a compilation timer, as seen in Figure 5.12, point 1. After a certain amount of time has passed, the Curator tries to delete the folder containing the K-P's partial compilation, the progress of which can be seen in Figure 5.12, point 2. To prevent this, the application provides the player with a button, seen in Figure 5.12, point 3, to move the folder to a different location on the real player's computer. To prevent the user from moving the folder too often, the button has a set cooldown during which it is not interactable. Alongside this, the application also displays a timer counting down to the deletion of the K-P's compilation folder, which is reset whenever the player moves the folder.

- **Deleting K-P**: If the player decides to delete K-P, the application features a timer counting down the compilation process, as seen in Figure 5.13, point 1. The player must stop the compilation by deleting three crucial files hidden in their real computer, which are revealed by performing specific actions in their computer's operating system. The application features three buttons that open the files' location, as seen in Figure 5.13, point 3. After deleting all three files, the final button opens a folder containing K-P.

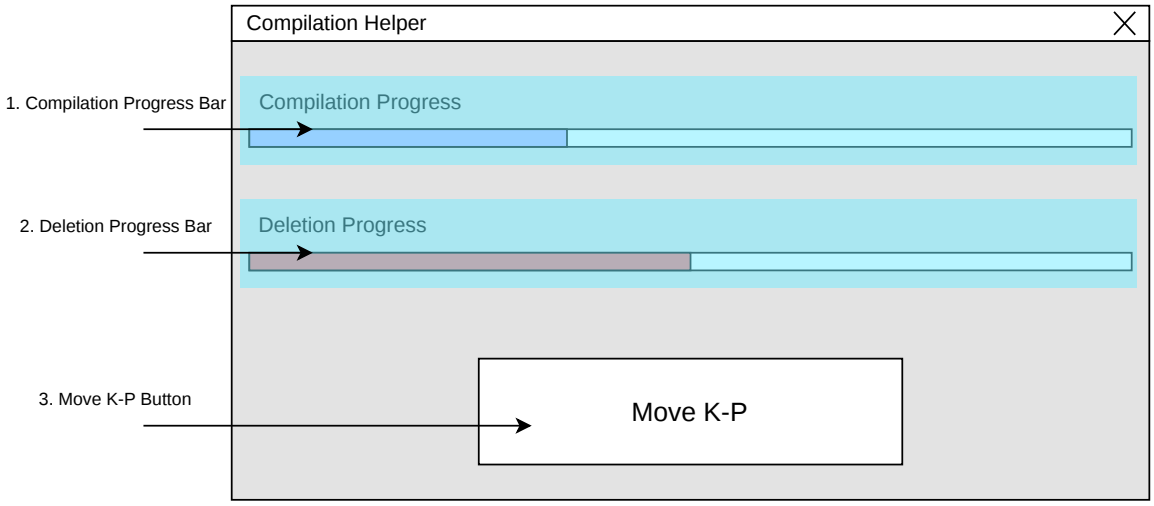


Figure 5.12: Wireframe of the Compilation Helper application's UI, when attempting to help K-P. 1. Progress bar showing the compilation progress. 2. Progress bar showing the K-P's deletion progress. 3. Button to move K-P's compilation folder to reset the deletion progress.

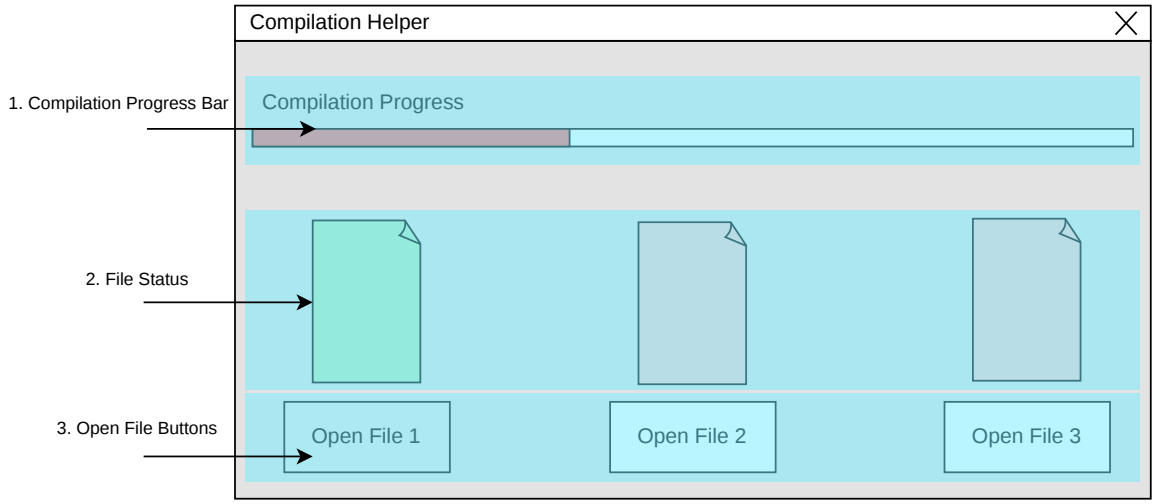


Figure 5.13: Wireframe of the Compilation Helper application's UI, when attempting to delete K-P. 1. Progress bar showing the compilation progress. 2. Visualization of the file deletion status. 3. Buttons to open the folder of the specific files.

File Uploader

During a specific narrative juncture, this application serves as a crucial bridge between the Simulated Layer and the System Layer. It includes a button, shown in Figure 5.14, point 2, to upload a file to the game. If the file's name and content are in the predetermined list, the application outputs a specific message to the user in the field shown in Figure 5.14, point 1; otherwise, nothing is output.

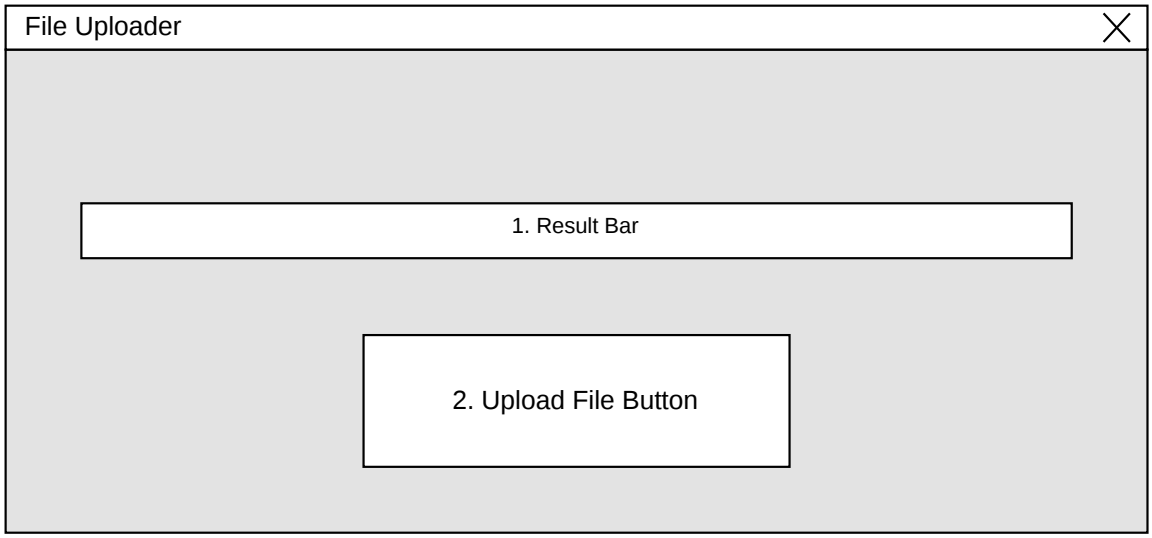


Figure 5.14: Wireframe of the File Uploader application. 1. Result text field. 2. Button used for uploading files.

Settings

This application allows the player to customize several properties of the Simulated Layer:

- **Wallpaper**: The user can choose a wallpaper from the predefined list by clicking the button shown in Figure 5.15, point 1.
- **Color Scheme**: The application features a way to customize the environment color scheme by interacting with the button as seen in Figure 5.15, point 2.
- **Sound Volume**: The application contains two sliders (seen in Figure 5.15, points 3 and 4) to alter the volume of the game's sound effects and its music.
- **Maximum Frames Per Second (FPS)**: To limit the computational overhead of the user's computer hardware, they can limit the application's frame rate by utilizing the provided slider, as seen in Figure 5.15, point 5.

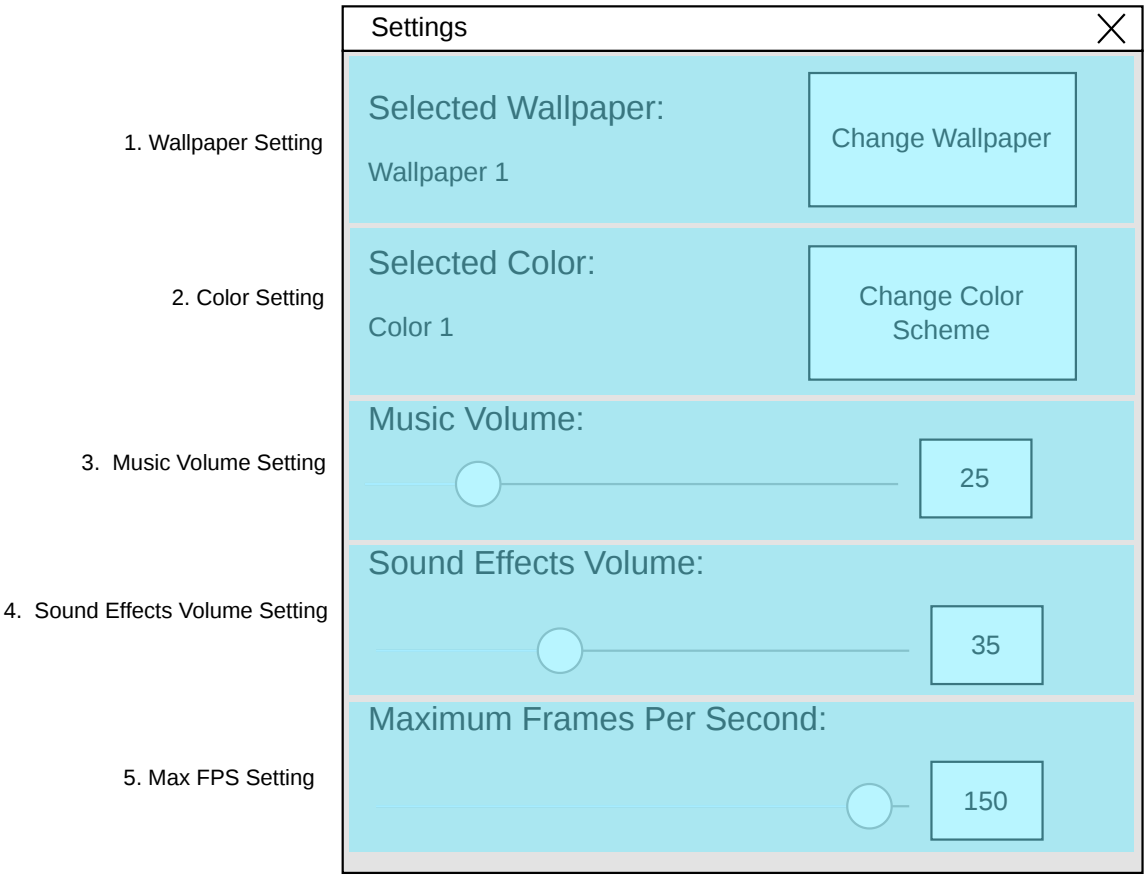


Figure 5.15: Wireframe of the Settings application. 1. Settings for changing wallpaper. 2. Settings for changing the color scheme. 3. Slider for changing the music volume. 4. Slider for changing the sound effects. volume. 5. Slider for limiting the rendered FPS.

5.2.2 The System Layer

Unlike the Simulated Layer, which confines the player's actions in a safe digital environment, the System Layer represents the actual user's operating system. In the context of *Kernel_Panic*, this encompasses the Windows environment, including its file system, external web browsers, and hardware states.

As the narrative escalates and K-P gains greater influence, the established safety net breaks down, forcing the player to treat their own operating system as an active extension of the game's world. This design choice aims to induce a sense of vulnerability and psychological distress.

By utilizing mechanics traditionally associated with malicious software, the game subverts the player's expectations of control. To progress throughout the narrative, the player can no longer rely solely on the simulated applications; instead, they must utilize their actual operating system to solve puzzles and uncover hidden files.

System State and User Information

To maximize the psychological impact, the game actively profiles the user by reading specific system information and monitoring hardware states. This gathered data is utilized to personalize the horror experience and create unconventional, system-level puzzles. The mechanics used are as follows:

- **Personalization**: At relevant narrative junctions, the game bypasses the user's chosen username and addresses them by the name saved in their operating system. Addressing the player directly by their name strips away the protective layer of anonymity. Similarly, the game reads and behaves differently depending on the user's computer time settings. This grounds the puzzle in the player's physical world.
- **Network Monitoring**: At a specific narrative point in the game, the player is required to disconnect their computer from the internet. The application can detect this disconnection state and proceed accordingly.
- **Audio State Detection**: In one of the game's most invasive mechanics, the player is required to alter their operating system's hardware state to progress. Specifically, the game silently monitors the system's master audio volume. To reveal hidden information, the player must digitally mute their computer's primary audio output.
- **Registry Intrusion**: The game directly interacts with the Windows Registry. By creating, reading, and reacting to specific changes within this registry, the game forces the player to navigate the core configuration of their operating system, significantly escalating the perceived danger.

The game requests this data and then continuously checks it against a predetermined condition. This is visualized in Figure 5.16.

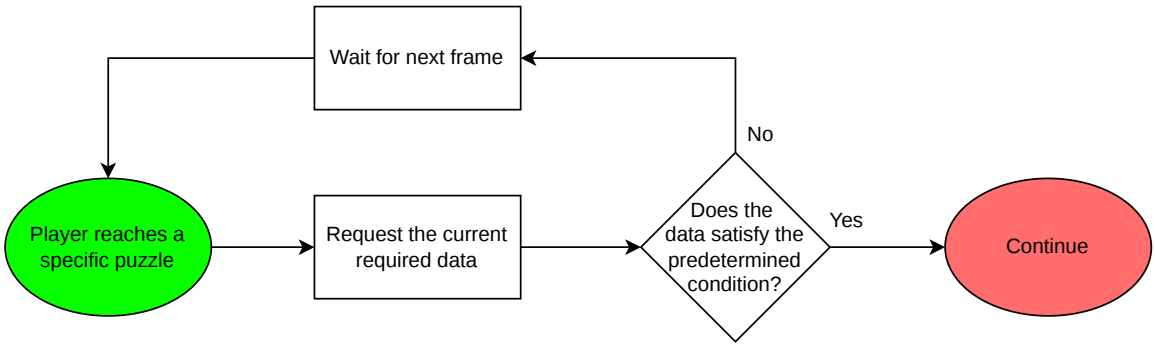


Figure 5.16: A visual representation of how the game uses the user's system states and information.

File Manipulation

Alongside conventional file manipulation, such as saving the player's progress to a special file, the game uses this concept to force the player to manipulate files on their operating system as a direct way to advance the game's narrative.

As a core gameplay mechanic, the player is frequently required to locate, read, alter, or permanently delete these generated files to progress. For instance, K-P might create text documents containing cryptic threats or mock system logs. The application silently monitors these specific external files in the background. Once the player performs the required action in their actual operating system, such as deleting a file representing a corrupted AI node, the game immediately detects this change and triggers the corresponding action.

Another notable example involves the game scanning all files and folders in a given directory, choosing a random one, copying its name, and appending *_Backup*. It then creates a new folder with this name, effectively disguising itself amongst the folder's content.

These mechanics bridge the gap between the simulated environment and the real world, heavily reinforcing the illusion of a malicious system threat.

Dialog Windows

To further blur the boundary between the application and the host operating system, the game actively uses native system dialog windows. Instead of confining all text and communication to the Simulated Layer, the narrative frequently spills over into these OS message boxes as visualized in Figure 5.17.

These windows are used to alert the player about a specific important event or as another way of communication from the game's NPCs.

From a psychological horror perspective, this method is highly effective. Users are conditioned to view native OS dialogues as authoritative and entirely separate from video game content. By hijacking this trusted interface, the game subverts this expectation and potentially induces fear that K-P is slowly taking control of their computer.

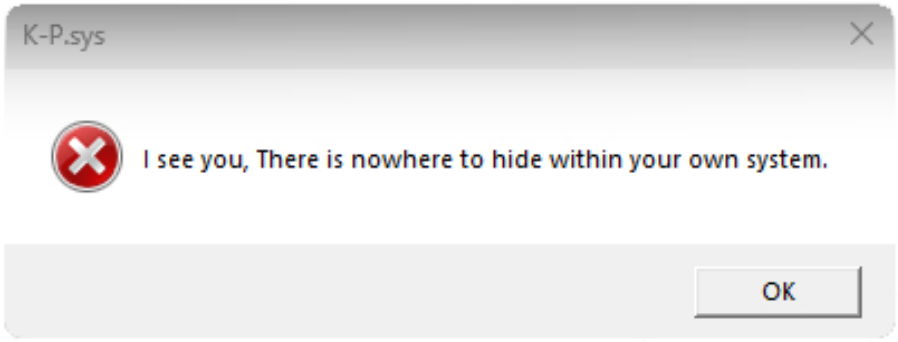


Figure 5.17: Screenshot of an example dialog window.

External Application

Another way to force player interactions outside the main application's window is by using a secondary application. This supplementary program serves as a pattern that the player must overlay over a row of numbers present in the main application's window. If overlaid correctly, the pattern reveals a secret code that further advances the narrative.

External Website

To extend the narrative beyond the local machine, the game incorporates a functional real-world website. This represents a definitive Alternate Reality Game (ARG) element, forcing the player to utilize their actual web browser to progress the game's narrative.

Rather than simulating a fake internet inside the game engine, the player is directed to a live URL hosted on the actual World Wide Web. The player is expected to interact with this site by uploading K-P's compiled and compressed files and downloading a specific file, which then must be imported back into the game using the File Uploader as described in section 5.2.1.

Chapter 6

Realization

This chapter details the technical realization of *Kernel_Panic*. While the previous chapter focused on the conceptual design and intended player experience, this section explores the concrete software engineering solutions utilized to bring these concepts to life.

6.1 Utilized Tools

For the development of *Kernel_Panic*, the Unity[1] video game engine was chosen. This engine was chosen for these reasons:

- **Ease of Use**: Unity engine is fairly simple to work with. The engine provides extensive documentation and a massive community facilitating efficient problem-solving. The GUI of this engine is very user-friendly and intuitive.
- **Programming Language C#**: This language is fully integrated within the engine, meaning it is very optimized and enables very complex solutions.

To create the game's UI, an online photo editing tool called Photopea[2] was used.

6.1.1 Unity Limitations

While Unity provided all the necessary tools for creating the Simulated Layer described in section 5.2.1, it is important to acknowledge that it is not the optimal engine for developing mechanics in the System Layer described in section 5.2.2. The engine lacks deeper, low-level integration with Windows.

During the development, a significant limitation of the engine was the lack of Windows-specific .NET libraries, most notably the `System.Windows.Forms`. This library is traditionally used in C# to easily invoke OS native dialog boxes and interact directly with the Windows environment.

Consequently, choosing Unity as the game engine meant that invoking actions in the System Layer had to be done using relatively limited Unity-supported C# libraries or by using Platform Invoke (P/Invoke) to call Windows API (WinAPI) functions. Nevertheless, overcoming these limitations demonstrates that even a game engine not designed for deeper OS integration can be engineered to deliver a game that requires it.

[1] Link to Unity's website: https://unity.com/
[2] Link to Photopea's website: https://www.photopea.com/

6.1.2 Utilization of Unity

Instead of relying on Unity's traditional 3D engine, the development of *Kernel_Panic* heavily utilized its robust 2D interface systems and asynchronous logic to construct the Simulated Layer. The following features were critical in the development:

- **Unity Canvas system**: The entire Simulated Layer was built upon Unity's Canvas system[3], acting as a primary plane upon which all graphical controls, such as text and buttons, were built. One possible narrative conclusion includes a scary face image sourced from Pixilart [12].
- **Game Objects**: The fundamental building blocks of any Unity application are *GameObjects*[4]. They act as base containers for all other components, ranging from UI elements and custom C# scripts to audio sources. In the context of the Simulated Layer, every window, button, and terminal line is fundamentally a *GameObject* manipulated via the engine.
- **C# Scripting (`MonoBehaviour`)**: While Unity provides the visual and audio building blocks, the core logic of *Kernel_Panic* is driven by custom C# scripts. By inheriting from Unity's base `MonoBehaviour`[5] class, these scripts are attached to *GameObjects* as components. They dictate the behavior of the Simulated Layer, manage the narrative state, and handle the critical interactions with the System Layer.
- **Coroutines**: Unity's approach to cooperative multitasking and time-based logic is handled through *Coroutines*[6]. While they execute on the main thread, they allow functions to pause execution and yield control back to the engine, resuming in subsequent frames. This behavior was used extensively in the project, most notably for simulating delays in writing messages to and from characters in-game.
- **Audio System**: The engine's built-in audio framework, utilizing *AudioSource* and *AudioClip*[7] components, was employed to manage the game's soundscape. This included UI feedback sounds, and background music, sourced from Freesound [1] [18] [8] and Pixabay [9] [19] websites.

The practical application of these systems, particularly the deep nesting of *GameObjects* within the *Canvas* to construct the complex UI of the Simulated Layer, is visualized in Figure 6.1.

[3] https://docs.unity.com/en-us/unity-studio/develop/ui-creator/canvas
[4] https://docs.unity.com/en-us/unity-studio/develop/gameobjects
[5] https://docs.unity3d.com/ScriptReference/MonoBehaviour.html
[6] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Coroutine.html
[7] https://docs.unity3d.com/6000.4/Documentation/Manual/AudioOverview.html
[8] Two sounds from this author were used and can be accessed using these links (*up.ogg*): https://freesound.org/people/stwime/sounds/545368/, (*up2.ogg*): https://freesound.org/people/stwime/sounds/545370/

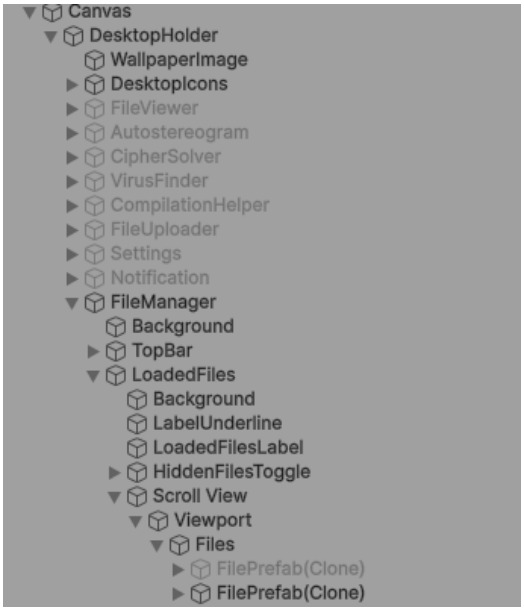


Figure 6.1: A view of the Unity Editor's Hierarchy window. It demonstrates how various applications and UI elements are heavily nested within the Canvas system using GameObjects to construct the Simulated Layer.

6.1.3 Scene Management

To ensure logical separation of certain game states, the game *Kernel_Panic* is divided into three distinct Unity Scenes:

- **Registration/Login Scene**: This initial scene serves as the entry point of the application. It handles the preliminary setup, including user profiling and initializing the core save file structure.
- **Main Desktop**: Serving as the core of the experience, this scene contains the entirety of the Simulated Layer described in Section 5.2.1. It houses the complex UI Canvas, all simulated applications, and the central narrative manager. The majority of the game's logic, including continuous background monitoring of external files and hardware states, is executed here.
- **User's Desktop Simulation**: In the fourth ending, as described in Section 5.1.3, the application briefly simulates the user's desktop within Unity's UI environment. Isolating this event prevents conflict with the game's main loop and provides a clean UI state.

The flow of transitioning between these scenes, including the asynchronous preloading of the final sequence to ensure a seamless experience, is illustrated in Figure 6.2.

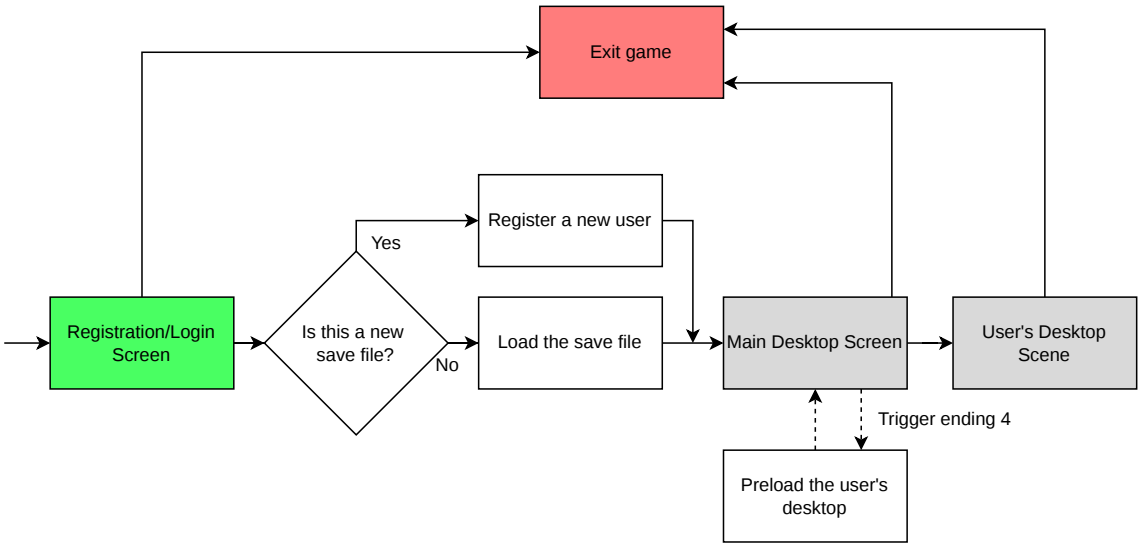


Figure 6.2: Flowchart detailing the scene transition logic in Unity. The diagram highlights the asynchronous preloading of the final ending scene while the main narrative continues to run within the Main Desktop environment.

6.2 Software Architecture: Modified MVC Pattern

To manage the entire codebase, which includes thousands of lines of code and their associated complexities, the application is built on a modified Model-View-Controller (MVC) design pattern, heavily supported by the Singleton pattern. This ensures separation of concerns by decoupling the visual representation, primarily used in the Simulated Layer, from the core game logic that manages the System Layer.

In this modified architecture, larger elements of the game, such as applications or the story manager, are divided into multiple independent modules. Each module utilizes its own isolated semi-MVC parts:

- **Views**: These components strictly handle the visual rendering of the user interface and capture user input. They are designed to remain entirely devoid of core game logic.
- **Models**: These represent the internal state and data structure of a specific module. They store the information necessary for the application to function independently of how that data is displayed.
- **Controllers**: Acting as the intermediary bridge, these components process incoming user input from the View, apply the necessary game logic, and update the Model accordingly.
- **Singletons (Managers)**: To facilitate communication between these isolated modules, without the need for tight coupling, each major game element employs a central manager designed as a Singleton. These managers expose specific controllers' APIs, allowing different systems (e.g., the narrative story manager and a simulated desktop application) to interact reliably without tight coupling.

To illustrate this data flow practically, consider the Chat Terminal module. When the player inputs a message, the View captures the string from the Unity Canvas and passes it to the Controller. The Controller processes this input, applies necessary logic, and updates the Model, which consists of pure C# classes storing the message history. Finally, the Controller signals the View to render the updated chat log. If an external system, such as the Story Manager, needs to trigger a new incoming message from, for example, the rogue AI, it interacts solely with the Chat Terminal's Singleton manager. This ensures the internal MVC structure remains strictly encapsulated and prevents direct interference with the UI components. This interaction is visualized in Figure 6.3.

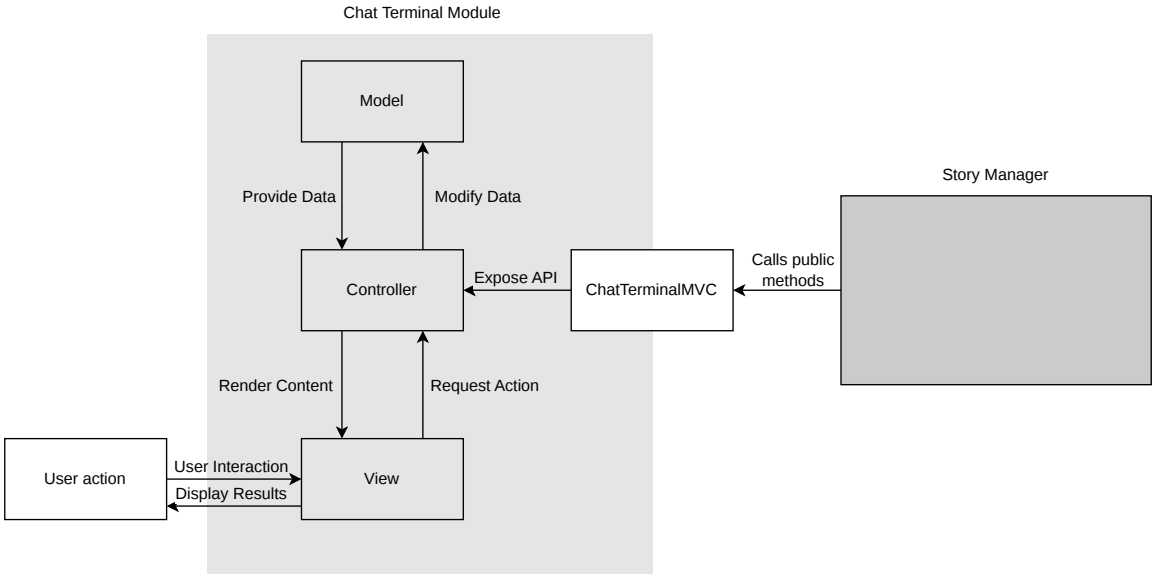


Figure 6.3: Diagram illustrating the Semi-MVC pattern utilized within the chat terminal module of the game. The Singleton Manager acts as a facade, exposing only the necessary Controller APIs to external systems while encapsulating the View and Model logic.

6.3 Implementation of the Simulated Layer

As outlined in Section 5.2.1, the Simulated Layer serves as the player's primary interface, mimicking a standard operating system. This section details the software engineering techniques and Unity systems utilized to construct this environment.

6.3.1 Desktop Interface

The foundation of the Simulated Layer is the simulated desktop environment, which acts as the primary hub for player interaction. Constructed entirely within Unity's UI Canvas, the desktop is divided into static background elements, interactive application icons, and a dynamic window management system.

To achieve an authentic operating system feel, several specific UI programming challenges had to be addressed:

- **Window Management**: Unlike traditional game UIs, *Kernel_Panic* requires its application windows to be layered over each other, as a real operating system would. This implementation is visualized in Figure 6.4.

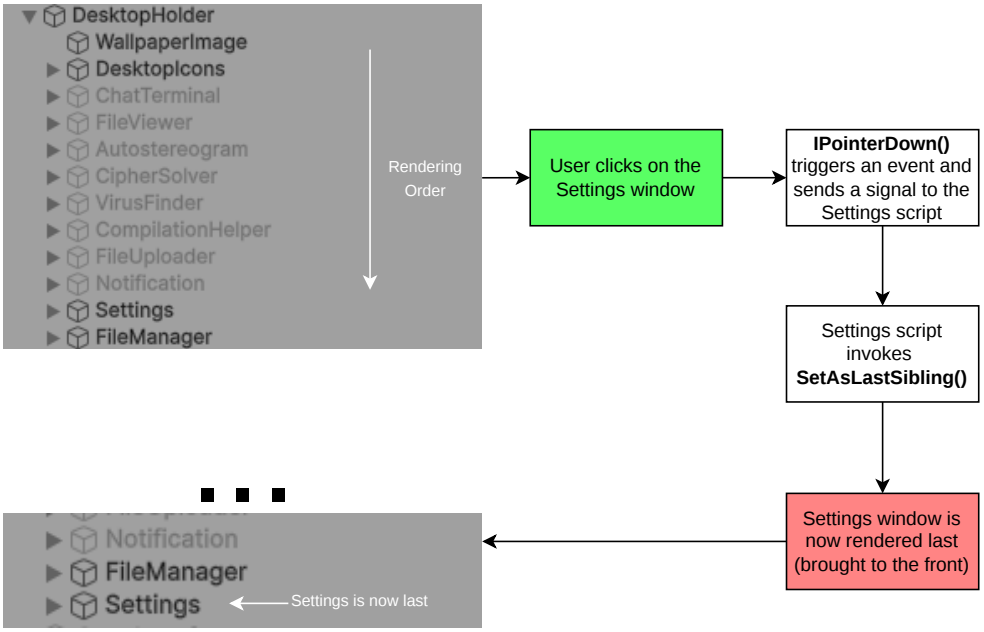


Figure 6.4: Diagram visualizing the functionality of switching the rendering order of the applications.

- **Draggable Windows**: A crucial aspect of OS simulation is the ability to freely move windows across the screen. This functionality was engineered by attaching custom C# scripts to the header area of each application window. These scripts implement Unity's built-in `IDragHandler` and `IPointerDownHandler` used for updating the window position.
- **The Taskbar**: Anchored to the bottom of the screen, the simulated taskbar provides persistent information. Notably, it features a simulated system clock. To further blur the line between the game and reality (as detailed in Section 5.2.2), this clock reads the host operating system's time via the `System.DateTime.Now` property. Alongside this clock, the bottom bar displays icons for currently open applications. Upon clicking any of these icons, the game immediately focuses the application, as explained in the first point.

6.3.2 Simulated Applications

To facilitate opening and closing different applications in the simulated desktop environment, two methods are available:

- **Keep Applications in Memory**: By preloading all of the possible applications, the game does not need to perform expensive tasks required for setting up the application's data, which saves CPU performance at the cost of higher memory usage throughout the game's runtime. The `SetActive()` is used for opening and closing of the applications.
- **Instantiate and Destroy**: The second method works by creating and destroying the application GameObject itself using the methods `Instantiate()` and `Destroy()`. This method reduces memory usage but introduces significant UI latency.

Due to the aforementioned unresponsiveness of the Instantiate and Destroy method, the game keeps the objects in memory throughout its runtime. This approach leverages the fundamental space-time trade-off concept in computer science, as described by Savage [15].

To prevent unexpected errors caused by file inconsistencies, the application must prevent the user from opening multiple instances of a single application. This is done by keeping a global list of all open application tags and opening only one if its tag is not already present. The logic is visualized in Figure 6.5. This tag is removed from the list upon closing of the application.

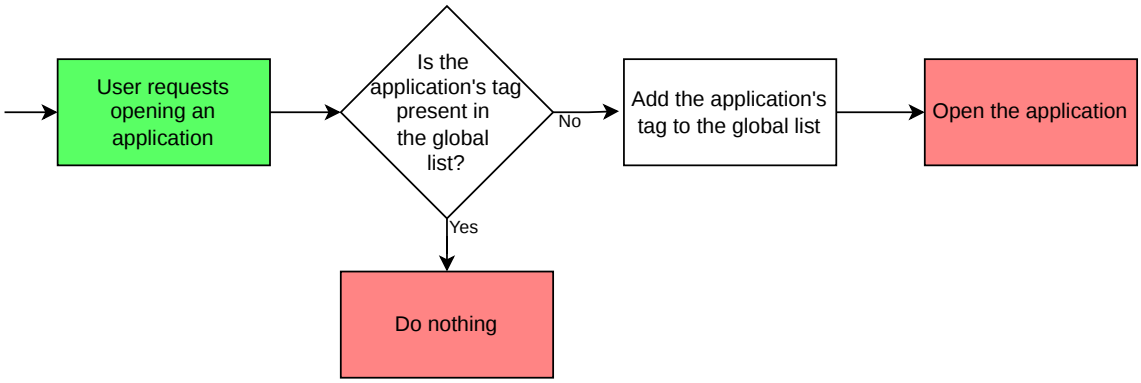


Figure 6.5: Visualization of the logic used for opening an application.

Chat Terminal

The Chat Terminal serves as the primary communication bridge between the player and the rogue AI. From a technical standpoint, developing this application required solving challenges in parsing and presenting text messages:

- **Parsing Dialogue Data**: To maintain a strict separation between the game's logic and its narrative content, all chat messages are stored externally in JSON (JavaScript Object Notation) format. When the terminal is initialized, the application reads these text assets and deserializes them into native C# data structures (such as lists or arrays of custom objects) using a JSON parser, which are then used for displaying specific messages on the application. For this process, the application employs an open-source library `Json.NET` [8].
- **Asynchronous Typing Simulation**: To reinforce the illusion of a living entity on the other side of the screen, the messages appear with a slight delay. This delay is calculated by $\frac{N}{v_{\text{typing}}}$, where N represents the string length and v_{typing} represents the specific user's typing speed. The messaging coroutine then pauses execution for the calculated delay.

File Manager

This application is another core component within the Simulated Layer. It allows the user to interact with available files. It works by preloading all file icons as disabled and enabling them with the `SetActive()` method. As mentioned at the start of this section, this approach saves CPU resources but incurs higher memory usage.

Interacting with the loaded file icons by clicking them with a mouse brings up the file's context menu. This menu allows the user to open the specified files with a specific application, such as the File Viewer. This is visualized in Figure 6.6

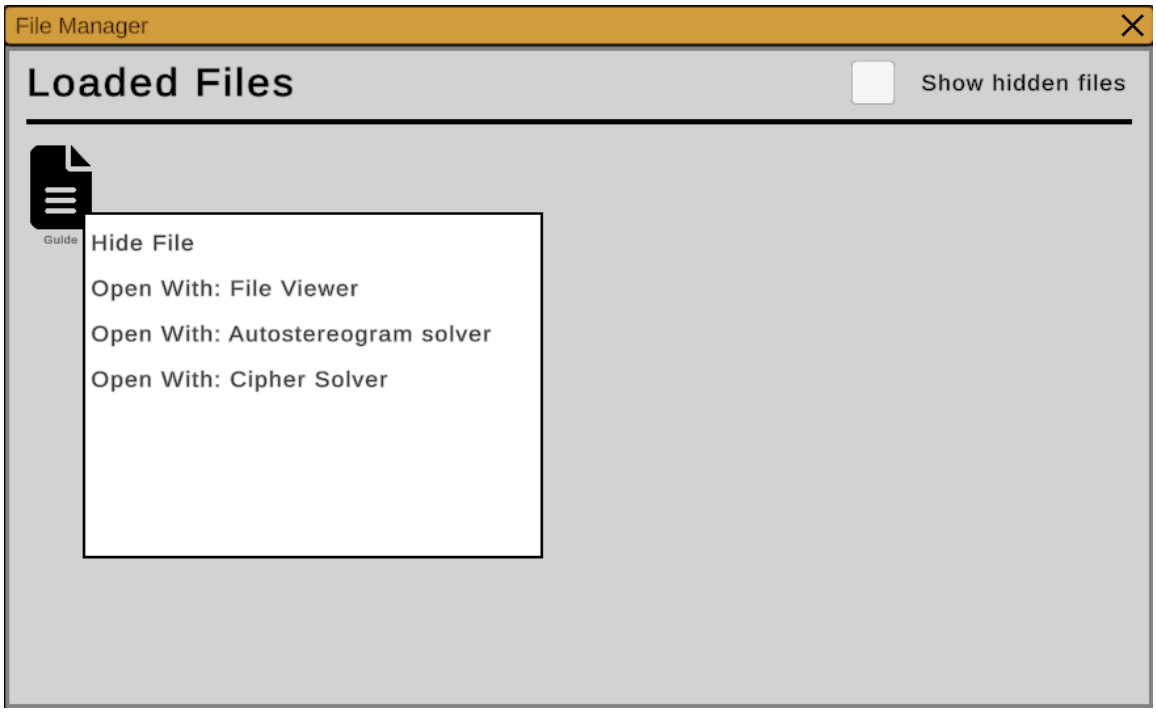


Figure 6.6: Screenshot of the game featuring the File Manager application with a single loaded file called Guide, alongside a context menu of the file.

File Viewer

Upon opening a file with the File Viewer application, the game renders the file's contents. These contents can be moved and scaled using either a mouse or the buttons as seen in Figure 6.7. This is achieved by modifying the `localScale` property of the file's contents. Alongside this, several of the files' contents include a text component that can be interacted with using a mouse. This is done by utilizing the `<link="linkName">` hypertext identifier.

By clicking the button to display the file's metadata, the application renders an overlay with additional information about the file (e.g., the author's name).

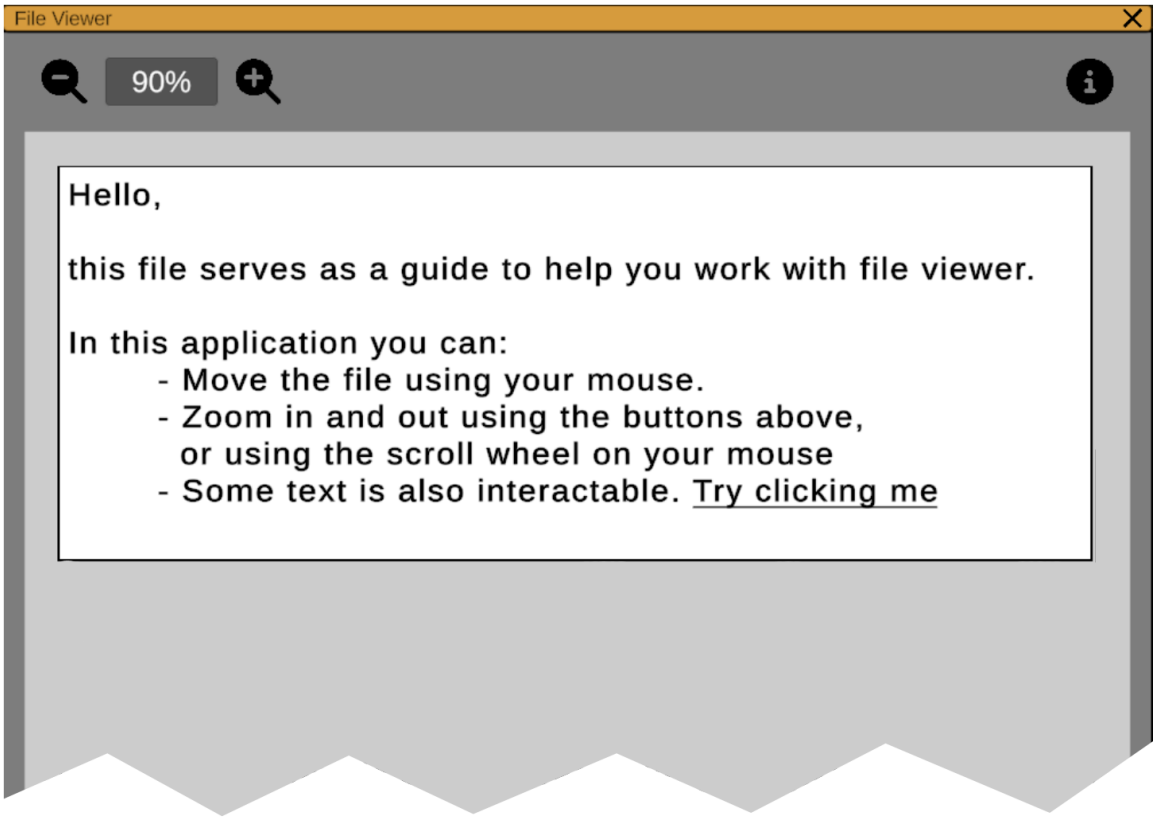


Figure 6.7: A digitally shortened screenshot of the game showcasing the File Viewer application containing the Guide file's contents.

Cipher Solver

Throughout the game, there are files that have encrypted content. This content can either be a text or an image. To encrypt these, the two cipher types used are the Vigenère Cipher, as described in section 4.4.2, and the XOR Cipher, described in section 4.4.3.

To decrypt these files, the player must find the correct cryptographic keys. The Cipher Solver application facilitates this process by providing a `TMP_InputField` for key entry and a dedicated UI button to execute the decryption algorithm, as seen in Figure 6.8. Upon successful validation, the decryption is performed, and the results are output instead of the file's original contents.

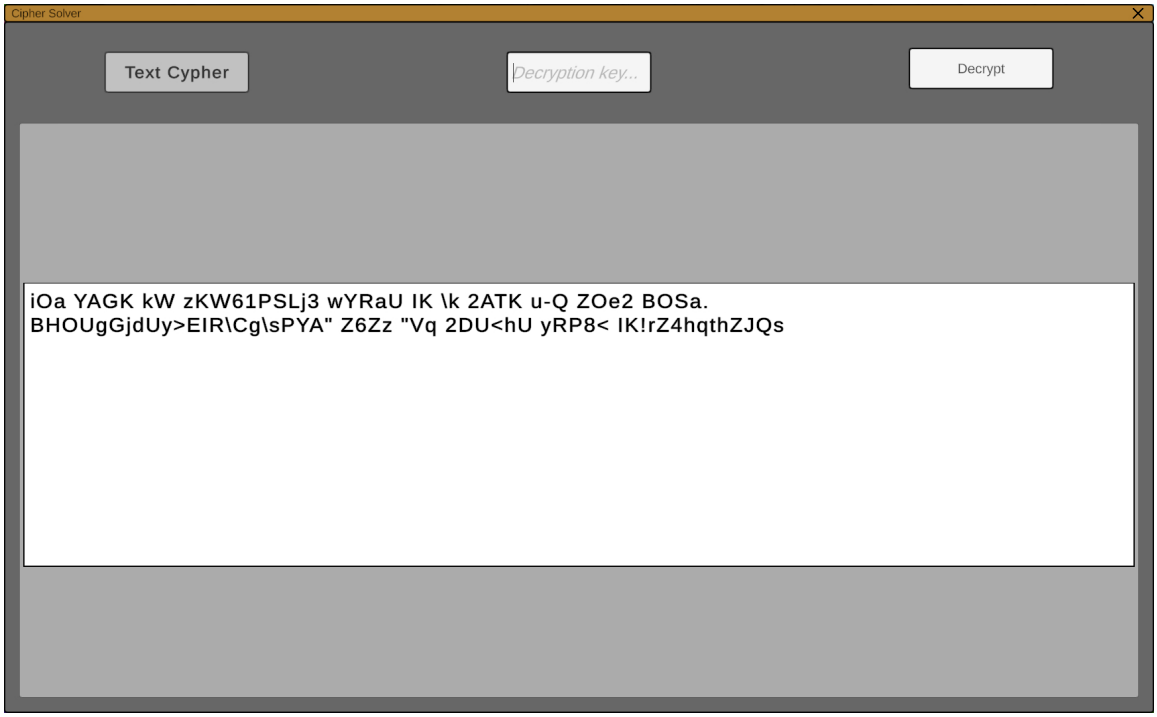


Figure 6.8: Screenshot of the Cipher Solver application featuring a file with encrypted text content.

Autostereogram Solver

To properly utilize the autostereogram puzzle in the game, the application needs to dynamically create the autostereogram images. This creation pipeline is described in Section 4.4.1.

The Autostereogram Solver application works by overlaying two identical images and, after each shift of one image, checking each pixel against the pixel beneath it. If the pixels' colors match, the pixel is coloured black. This algorithm is visualized in Figure 6.9. The resulting application is displayed in Figure 6.10.

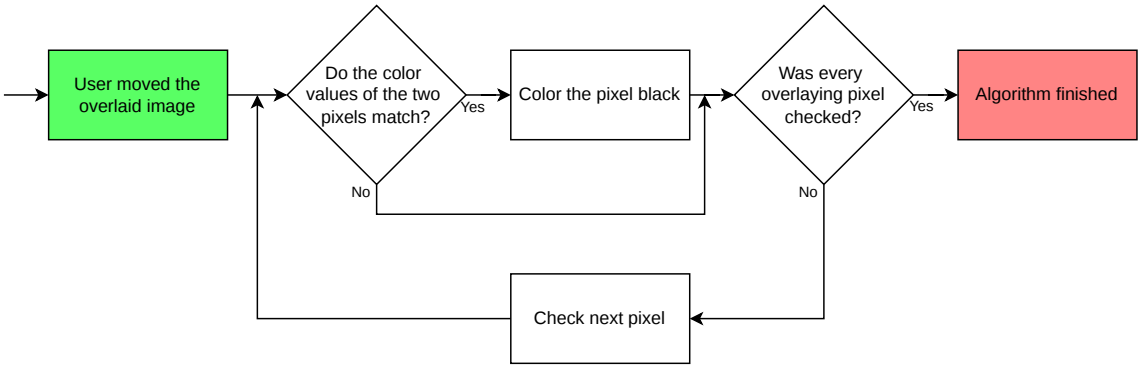


Figure 6.9: Diagram visualizing the logic behind the autostereogram solver.

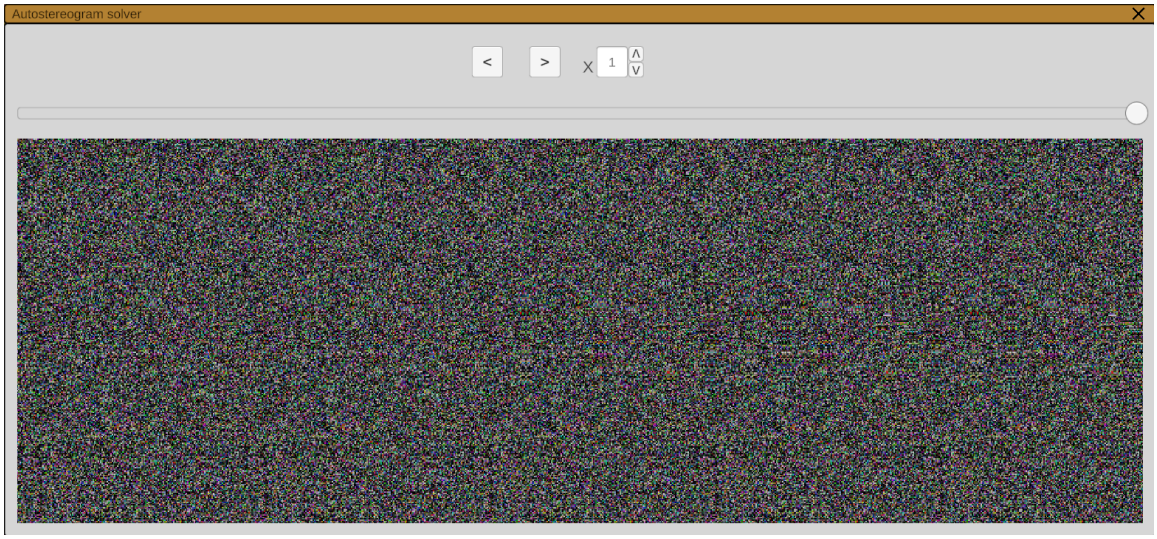


Figure 6.10: A screenshot of the Autostereogram Solver application.

Virus Finder

At a predefined narrative juncture, the game creates a random number of viruses in the Simulated Layer. These are visualized as hidden desktop icons that the user must delete to progress, as seen in Figure 6.11. To expose these files on the virtual desktop, the user must run a virus scan using the Virus Finder application.

Virus Finder simulates its virus search by rendering a progress bar, whose speed is determined by the number of generated viruses. The filling of the progress bar, seen in Figure 6.12, is implemented using Unity's *Coroutine* system, which does not block Unity's main thread.

After the application completes its search, it reveals any hidden viruses on the desktop using `SetActive()`.

Figure 6.11: Icon of the virus present on the virtual desktop.

Figure 6.12: A screenshot of the Virus Finder application during the process of searching for viruses.

Compilation Helper

During the ending sequence, K-P will try to compile itself for upload to the internet. The user utilizes the Compilation Helper to monitor and either help or hinder this process. This choice will lead to the application itself featuring a different UI. By choosing to help K-P, the application will feature the UI shown in Figure 6.13, and, conversely, by helping the Curator, the application's UI will change to the one shown in Figure 6.14.

By design, the application features a streamlined user interface consisting primarily of status indicators and progress bars. Consistent with previously described applications, the continuous updating of these progress bars is handled asynchronously via Unity's *Coroutine* system to ensure smooth visual feedback without blocking the engine's main thread.

Architecturally, this application features a lightweight UI and relies heavily on mechanics, such as file and registry manipulation and audio state detection, as described in the System Layer (see Section 6.4).

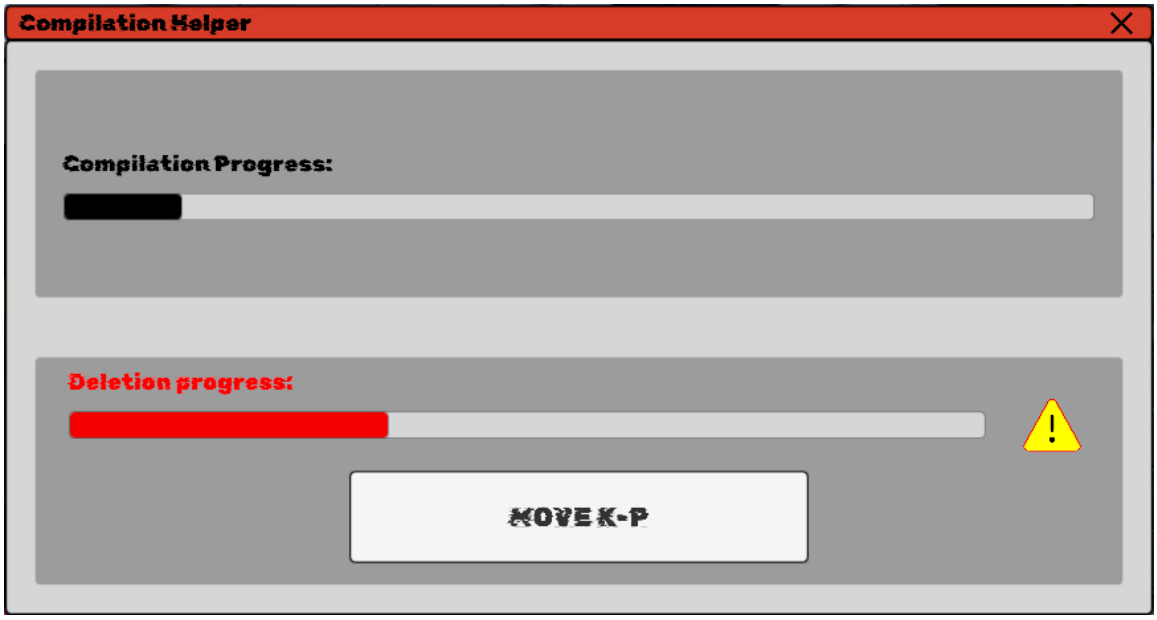


Figure 6.13: A screenshot of the Compilation Helper application when deciding to help K-P escape.

Figure 6.14: A screenshot of the Compilation Helper application when deciding to help the Curator delete K-P. When a file is deleted, the UI will update, as seen in the *compiled_core_logic.bin*.

File Uploader

By uploading specific known files, the application can output a predetermined string of text, as seen in Figure 6.15.

This is implemented by keeping a list of known file names and their contents alongside the output result text.

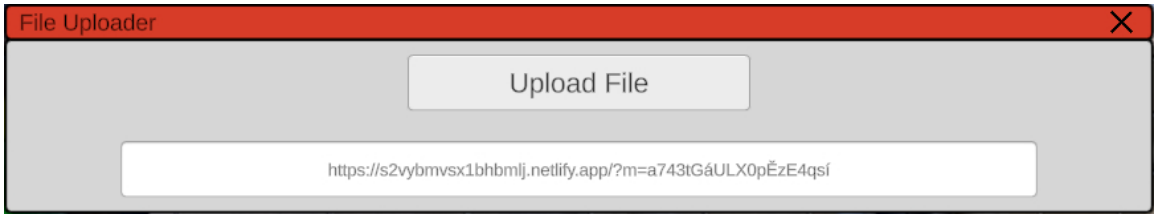


Figure 6.15: Screenshot of the File Uploader Application while uploading a valid file and presenting its result.

Settings

From a technical standpoint, the application primarily utilizes Unity's standard UI components, such as `Slider` and `Toggle`, seen in Figure 6.10, to capture user input. The application features three core customization options:

- **Wallpaper Selection**: The user can set the virtual desktop's background by choosing from a preloaded array of images.
- **Colour Palette Customization**: To facilitate visual customization, the application incorporates a colour picker tool. Rather than developing a redundant custom solution for a standard UI element, an open-source colour picker asset [2] was integrated into the project. This feature allows the player to freely adjust the simulated operating system's colour scheme.
- **Audio Configuration**: When the user adjusts the volume sliders, the application's controller intercepts these linear floating-point values and maps them to the exposed parameters of Unity's `AudioMixer`. Because human hearing perceives sound intensity logarithmically, the linear slider values are mathematically converted into a logarithmic decibel (dB) scale before being applied to the respective mixer groups (e.g., Master, Music, and SFX) using the following equation:

$$V_{dB} = 20 \cdot \log_{10}(V_{linear}) \tag{6.1}$$

Conversely, to ensure the UI accurately reflects the active configuration upon initialization, this conversion is also performed in reverse, translating the mixer's logarithmic values back into the linear space required to position the slider handles correctly:

$$V_{linear} = 10^{\frac{V_{dB}}{20}} \tag{6.2}$$

To provide adequate volume overhead, the slider allows a maximum linear value of 2, which corresponds to approximately +6 dB. Furthermore, to prevent mathematical undefined behavior (since the logarithm of zero is undefined), the minimum linear value is explicitly clamped to 0.0001, which corresponds to −80 dB. In Unity's audio engine, this specific value mutes the audio output.

- **Maximum Frames Per Second (FPS)**: By interacting with the Maximum FPS Slider, the user is able to change the maximum frame rate of the application. This artificial limitation is present to limit unnecessary computational overhead. By preventing the Graphics Processing Unit (GPU) from rendering an excessive number of unperceivable frames, the application reduces overall power consumption and guarantees a more consistent frame-time delivery.

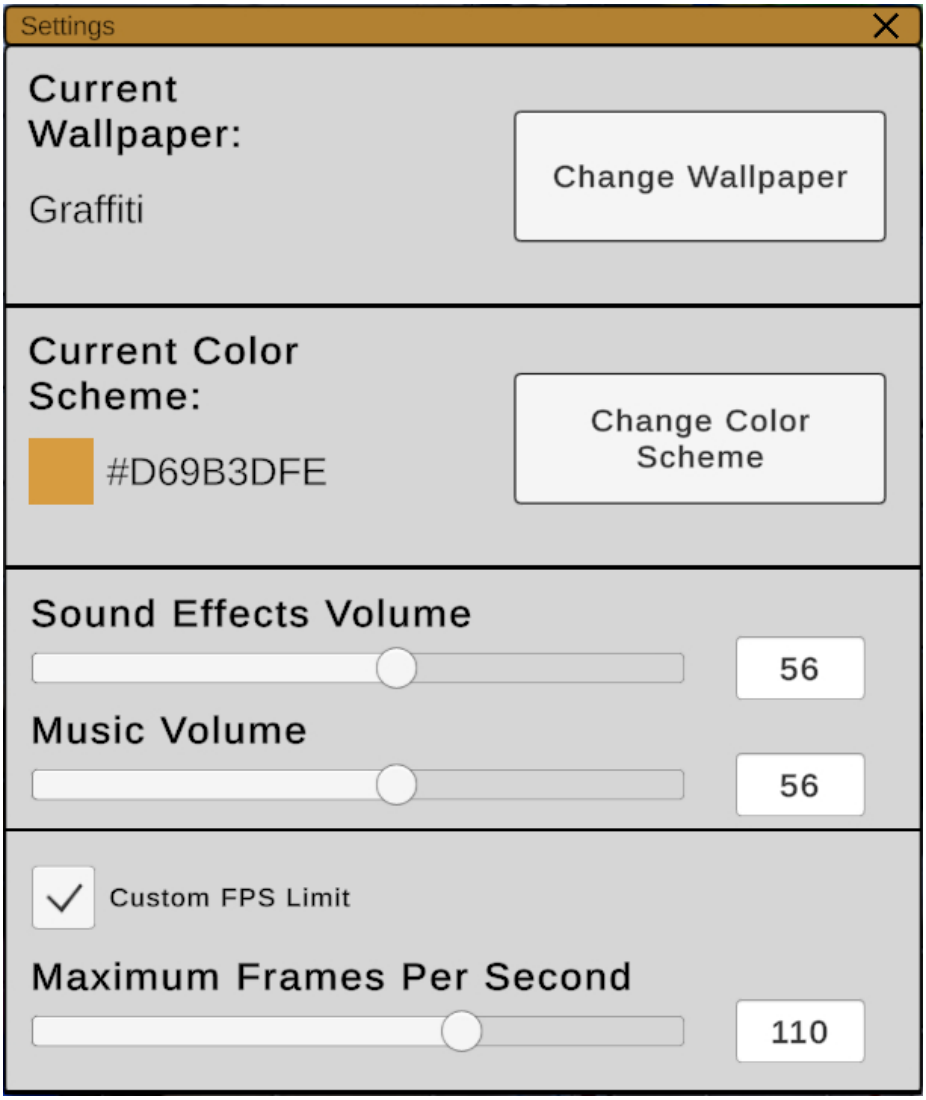


Figure 6.16: Screenshot of the Settings application.

6.4 Implementation of the System Layer

While the Simulated Layer provides the visual interface and confines the player within a virtual sandbox, the System Layer represents the core technical innovation of *Kernel_Panic*. This layer is responsible for breaking the fourth wall by escaping the Unity engine's standard boundaries and interacting directly with the player's real host operating system (Windows).

Architecturally, the System Layer relies heavily on the underlying C# .NET framework rather than Unity-specific APIs. This section details the specific software engineering techniques used to achieve this integration, described in Section 5.2.2.

6.4.1 Windows API and Platform Invoke

To achieve true integration with the host operating system, the game must transcend the limitations of Unity's managed .NET sandbox. In the .NET ecosystem, *managed code* refers to code that executes under the supervision of the Common Language Runtime (CLR). The CLR provides high-level services such as automatic memory management (Garbage Collection), type safety, and security boundaries [6][9]. Conversely, *unmanaged code* executes directly on the operating system outside the CLR's control, typically written in C or C++, and requires manual memory management.

While standard managed C# libraries offer basic file manipulation, advanced system-level interactions, such as manipulating the user's real desktop environment, require direct communication with the unmanaged Windows Application Programming Interface (API).

This bridge between the game's managed C# environment and the unmanaged C/C++ code of the Windows OS is achieved using Platform Invoke (commonly referred to as P/Invoke) [6][10]. By utilizing the `System.Runtime.InteropServices` namespace, the game can import and execute native functions residing within core Windows Dynamic Link libraries (DLLs). This concept is visualized in Figure 6.17.

[9] Managed code documentation: https://learn.microsoft.com/en-us/dotnet/standard/managed-code

[10] P/Invoke documentation: https://learn.microsoft.com/en-us/dotnet/standard/native-interop/pinvoke

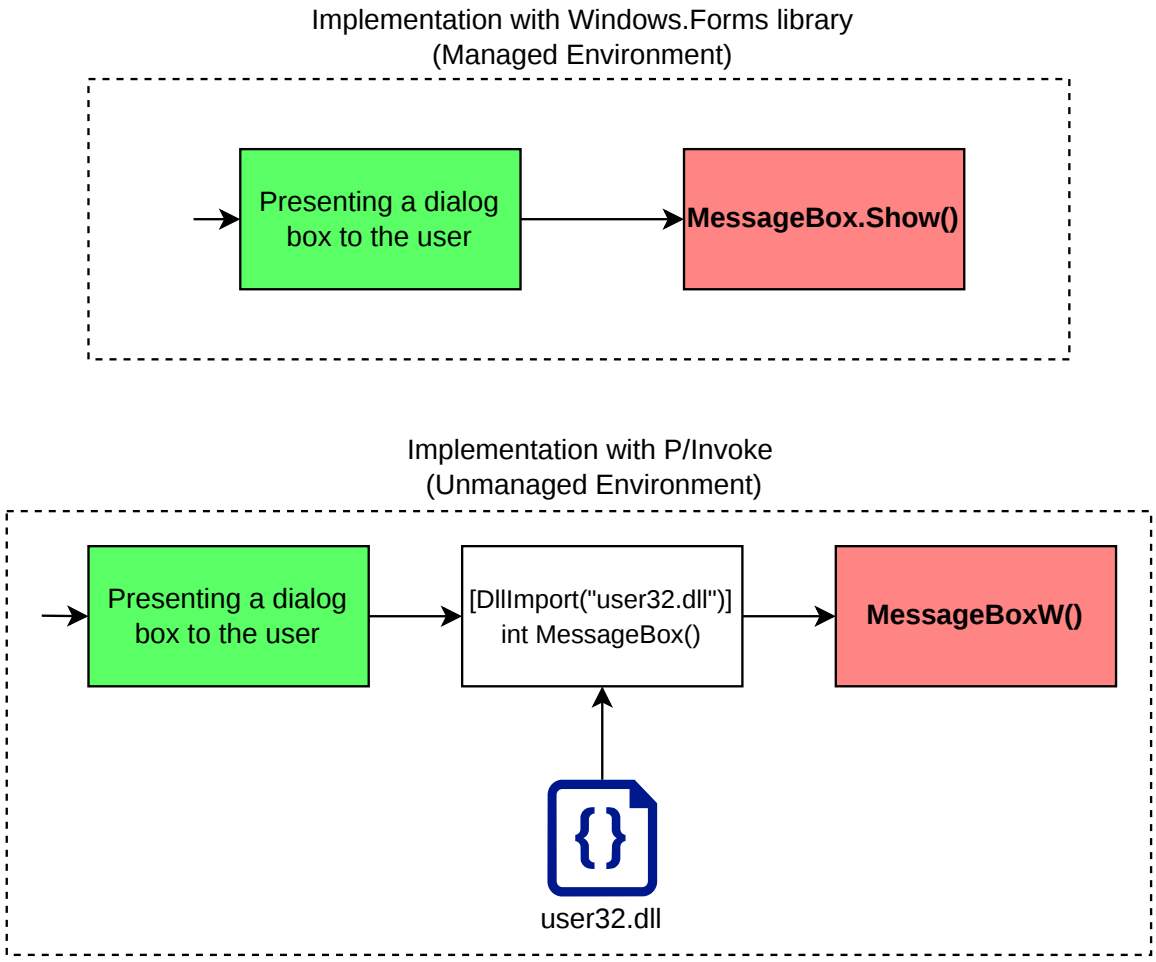


Figure 6.17: A conceptual comparison between invoking a Windows dialog window via a managed .NET wrapper (e.g., `Windows.Forms`) and directly executing the native Windows API function via P/Invoke.

6.4.2 Host Environment Manipulation

The most fundamental level of interaction with the player's real operating system involves reading the host environment's state. By extracting genuine system metrics and user data, the game immediately breaches the fourth wall, blurring the boundary between the simulated desktop and the physical machine.

User Information

A signature fourth-wall-breaking mechanic is to address the player by their real name, rather than the username they chose. This is done using P/Invoke (see Section 6.4.1) and importing the `GetUserNameEx()` method from the *secur32.dll* library, which returns the player's name stored on the user's computer.

System State

Another method of anchoring the game within the player's physical reality involves reading and modifying numerous system states:

- **Network State Monitoring**: The application is capable of detecting whether the user is connected to the internet. This is done by utilizing the `Ping` class, visualized in Figure 6.18.

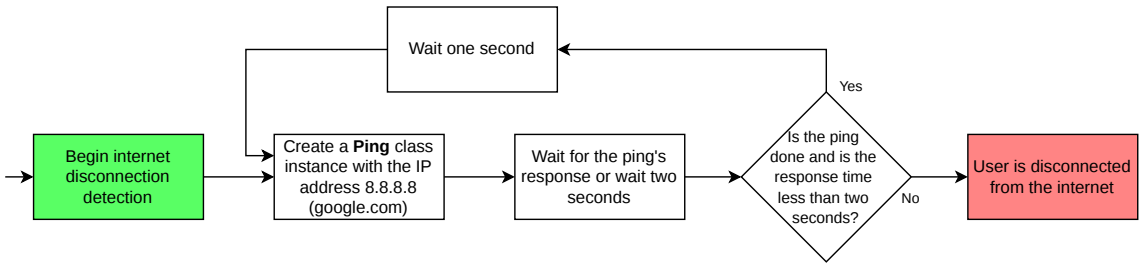


Figure 6.18: Logic of internet disconnection detection. The logic continuously pings *google.com*, and if no response arrives within 2 seconds, the system assumes the user is disconnected from the internet.

- **Audio State Detection**: The game is capable of reacting to the player's audio output state. Because querying the *Windows Core Audio APIs* directly from managed C# code is not natively supported in Unity due to the underlying COM (Component Object Model) architecture, a custom native DLL plugin was developed in C++. This unmanaged plugin acts as a bridge to safely query the system's default audio endpoint. The resulting mute state is exported (using `__declspec(dllexport)`) and seamlessly read by the Unity engine via P/Invoke (see Section 6.4.1).

  The detection logic itself runs in a continuous loop that checks the audio device's digital state on every frame.

- **Registry Manipulation**: Another interactive puzzle involves forcing the player to manually edit the host's system registry. The game creates a custom registry key[11] and requires the player to either modify its value or delete the entry entirely to progress. This interaction is facilitated by the `Registry` and `RegistryKey` classes, which provide the managed .NET methods required to securely create, modify, read, and delete the entry and its associated values.

Clipboard Manipulation

At several narrative junctures, the game uses Windows' clipboard to either advance the narrative or assist the user with certain tasks. This functionality is implemented using Unity's native `TextEditor` class to create the text in memory, and the methods `SelectAll()` and `Copy()` to copy it to the clipboard.

[11] Specifically located under the `HKEY_CURRENT_USER\Software` directory.

Simulated Keyboard Input

A narrative sequence requires the player to interact with their files, while the game's character tries to prevent that. The game simulates this by invoking the *Win + D* keyboard combo, which minimizes all opened applications. This mechanic is implemented by invoking the `keybd_event()` function, which is accessible by importing the *user32.dll* by using P/Invoke as described in Section 6.4.1.

6.4.3 File Manipulation

One of the most psychologically impactful methods of breaking the fourth wall involves manipulating the user's actual files on their computer.

The creation logic for several important files is implemented using the robust `System.IO` namespace, as shown in Figure 6.19. Hardcoding paths into the code would break the application right away due to possible naming differences across computers; therefore, the game dynamically resolves target directories (such as the Desktop) during runtime using `System.Environment.GetFolderPath`.

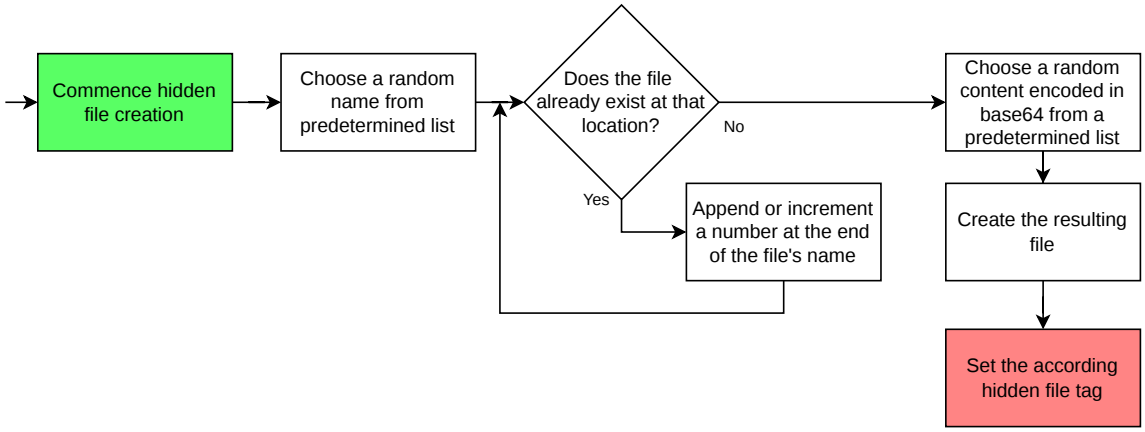


Figure 6.19: Logic of the creation of important files. The file's contents are sentences from a predetermined list, which are then encoded to base64. It is not necessary to understand the content of these files to progress the narrative.

Besides the more unconventional file manipulation, the game also uses a more traditional approach when creating a save file. The functionality of this file is crucial due to the nature of the game. The location of this file corresponds to `Application.persistentDataPath` (which practically leads to `AppData/LocalLow`). The implementation of this data persistence is visualized in Figure 6.20.

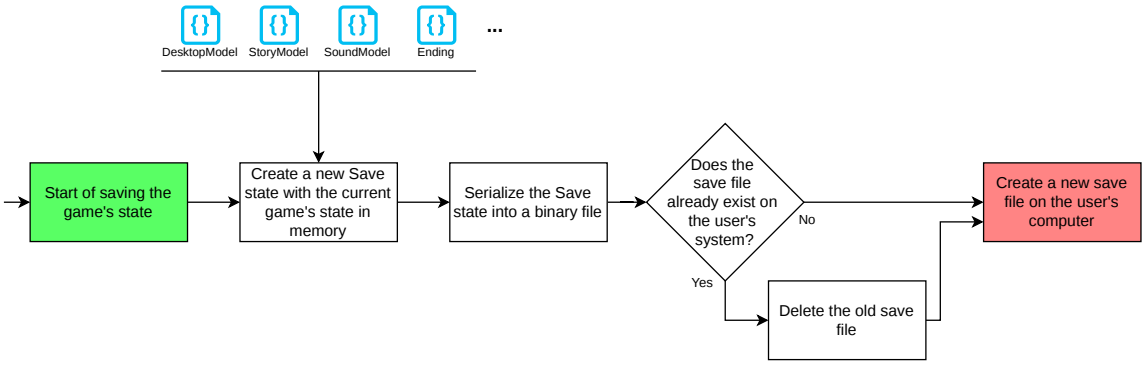


Figure 6.20: A flowchart visualizing the game's saving logic. By marking core data models as *Serializable*, the application aggregates the current state, serializes it into a binary format, and safely writes it to a physical file on the host computer, proactively replacing any existing save data.

Certain game mechanics require the player to explicitly select files on their computer. A significant technical limitation of the Unity engine is the lack of a built-in file browser API for compiled builds; standard methods such as `EditorUtility.OpenFilePanel()` function only within the Unity Editor and causes crashes in the final application.

To overcome this limitation, the application integrates the open-source library called `UnityStandaloneFileBrowser` [3]. This third-party library provides a robust, managed C# wrapper that safely and reliably invokes the host operating system's native file selection dialogs.

Another mechanic in the game is the compilation of K-P. This process ends with K-P being compressed into a single zip file.

6.4.4 Dialog Windows

As described in Section 5.2.2, the game utilizes native system dialog windows. By importing the `user32.dll` library using P/Invoke (as described in Section 6.4.1), the application is able to call the `MessageBoxW` method with adequate parameters to invoke a native Windows dialog window.

6.4.5 External Application

The game requires the player to manually run a provided standalone application, `Encryption Pattern.exe`, developed with the Windows Presentation Foundation (WPF) framework, to solve a specific puzzle. Upon launch, this executable leverages WPF's native, hardware-accelerated UI composition to render a borderless, resizable window displaying a cryptographic pattern with transparent cutouts (as shown in Figure 6.21a).

To solve the puzzle, the player must physically drag and superimpose this external window over the active game client. When correctly aligned, the transparent spaces within the pattern act as an optical stencil, revealing a hidden code, essential to advancing the narrative.

A significant UX (User Experience) challenge in this design stems from the game's support for dynamic window sizing and varying monitor resolutions. For the optical stencil to function correctly, the external pattern must be manually scaled by the player to perfectly match the game's current rendering space. To facilitate this, the in-game UI provides a

dedicated visual anchor (a calibration marker), seen in Figure 6.21b. The player must resize the external application until its dimensions align with this in-game anchor, ensuring accurate visual registration between the two independent applications.

(a) The image pattern utilized in the standalone `EncryptionPattern.exe` application. It features three corner cutouts that perfectly align with the corresponding anchor points in the game. To assist the player in finding the correct position on the in-game number grid, the overlay includes a visual hint indicating the location of the grid's single zero.

(b) The in-game calibration grid rendered within the Unity engine. This interface serves as the foundational anchor, featuring corresponding corner points for easier alignment. It provides the visual reference the player needs to accurately scale and position the external optical stencil.

Figure 6.21: The images used within the external application and the game itself, serving as an image puzzle.

6.4.6 External Website

As described in Section 5.2.2, the application employs a functional real-world website.

From a technical standpoint, the external website's interface is deliberately lightweight and built with standard front-end web technologies, specifically HTML and JavaScript. This ensures broad compatibility and immediate responsiveness across different desktop browsers without the need for complex backend infrastructure.

To ensure high availability and global accessibility, the application is hosted on the Netlify platform [12].

The website needs to check the contents of uploaded ZIP files to confirm their validity. This is done entirely client-side by integrating the open-source `JSZip` library [5].

The logic of the website is visualized in Figure 6.22. The value of the `m` parameter key is the real user's name (see Section 6.4.2) and is encrypted using a Vigenère cipher. It is validated with the following process:

- **Decryption Using a Vigenère Cipher**: The parameter value is first decrypted using the same cipher key and alphabet as was used for encryption.
- **Length Validation (Prefix)**: The first four characters of the decrypted string are parsed as an integer, representing the exact character length of the user's name.
- **Payload Extraction**: Based on the verified length, the subsequent characters are extracted. This substring represents the actual narrative payload (the user's real name).
- **Integrity Verification (Suffix)**: The final ten characters of the string act as a security checksum, containing a truncated SHA-256 hash, described in Section 4.4.4, of the user's name. The web client independently computes the SHA-256 hash of the extracted payload and compares it against this suffix.

[12] Netlify's URL: https://www.netlify.com/

This implementation ensures protection against manual URL tampering. It effectively prevents players from bypassing the local gameplay loop, as they cannot forge a valid URL parameter without reverse-engineering the cipher key and hash logic directly from the game.

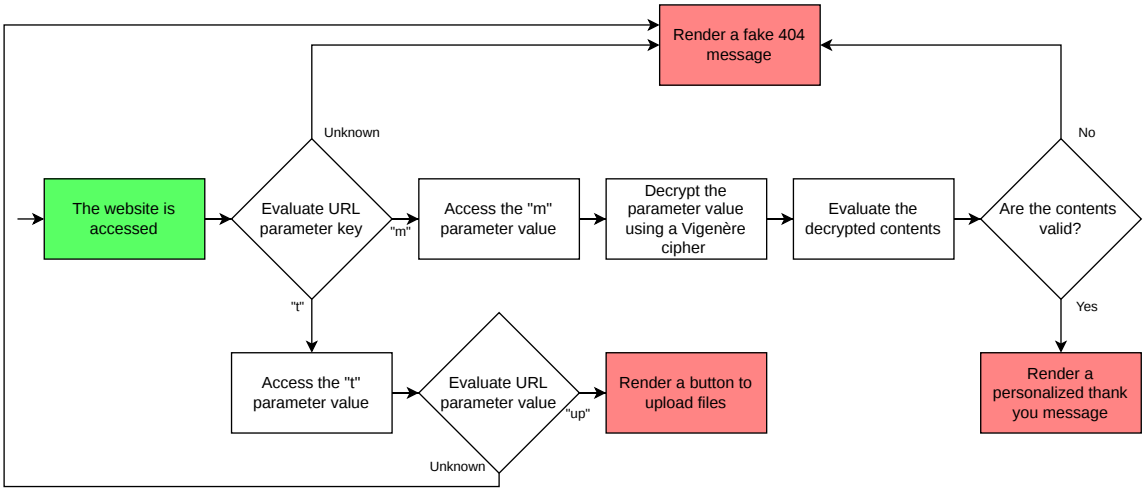


Figure 6.22: Visualization of the external website's logic. Upon access, the system evaluates the URL parameter keys and cryptographically validates their values, rendering the appropriate narrative content or a simulated 404 error.

Chapter 7

Testing and Evaluation

To ensure both the technical stability of the application and the effectiveness of its narrative design, the testing phase was divided into two primary categories: internal system testing (focusing on code stability and OS integration) and user playtesting (evaluating the psychological impact and puzzle difficulty).

7.1 Functional and System Testing

Due to the unconventional nature of *Kernel_Panic* and its reliance on the Windows native API, special consideration was given to evaluating its safety.

7.1.1 Testing During Development

The majority of the functional testing was conducted iteratively alongside the development of specific mechanics. Given the project's heavy reliance on low-level system interactions, a continuous testing approach was critical to prevent catastrophic engine crashes or host system instability.

7.1.2 Debugging and Profiling

To maintain the stability of the Simulated Layer, standard debugging tools, such as the Rider IDE debugger, were used continuously to trace logic flaws. Furthermore, the Unity Profiler was periodically employed during the implementation of asynchronous routines (Coroutines) and background processes. This ensured that features such as the continuous audio monitoring loop did not cause memory leaks or degrade the application's overall framerate.

7.1.3 Operating System Compatibility

To ensure the application functions reliably across different user setups, compatibility testing was conducted across three distinct Windows environments. This approach verified not only API consistency but also hardware performance and dependency resolution.

Primary Workstation (Windows 11) Alongside the development itself, the initial and continuous testing were performed on the main development machine running Windows 11. This served as the baseline to ensure that all native P/Invoke calls, window management

mechanics, and the custom C++ audio plugin functioned correctly under optimal hardware conditions.

Low-Specification System (Windows 10) To evaluate backward compatibility and performance under hardware constraints, the application was tested on a secondary, lower-tier computer running Windows 10. This test was crucial to verify that the continuous background monitoring loops did not overwhelm older hardware. Furthermore, it confirmed that legacy versions of the Windows native API handled the desktop manipulation commands (such as file creation and window forcing) identically to Windows 11.

Pristine Virtual Machine (Windows 11) The final and most critical compatibility test was executed on a freshly installed Windows 11 Virtual Machine (VM). Development computers inherently contain pre-installed dependencies (such as specific .NET frameworks or Visual C++ Redistributables) that standard user machines might lack. Testing in a pristine VM sandbox guaranteed that the compiled standalone build of *Kernel_Panic* was fully self-contained. It successfully proved that the application could execute its fourth-wall-breaking mechanics on a completely new system without crashing due to missing external libraries.

7.2 User Playtesting

The success of a meta-horror game relies entirely on the player's subjective experience. To evaluate this, a series of playtesting sessions was conducted with a small group of independent testers.

7.2.1 Testing Methodology

A group of independent testers was recruited via an unofficial Discord server for the Brno University of Technology Faculty of Information Technology. This approach ensured a sample of players who are generally familiar with standard video game tropes and semi-advanced computer manipulation, making them the ideal target audience for a meta-horror experience designed to subvert those exact expectations.

Unsupervised Remote Playtesting The application was distributed as a compressed file for the participants to play independently on their personal computers. This remote testing methodology was deliberately chosen to preserve the authenticity of the horror experience. For a game that relies on invading the user's private digital workspace, isolating the player in their natural environment, without a developer observing them, is critical for eliciting genuine emotional responses and moments of surprise.

7.2.2 User Feedback

The playtesters were subsequently asked about their experience of playing the game via a Google Forms document[1].

The survey received seven responses, providing qualitative insights and sufficient preliminary data to evaluate the core mechanics, puzzle difficulty, and overall user experience.

[1]Feedback form: https://docs.google.com/forms/d/e/1FAIpQLSe_Lwf5RQxr-dyE15XTRSEx2kK0c74lT-zRC1BxCzY6HLNV7g/viewform?usp=dialog

Playtime and Narrative Choices

Interestingly, there was no single definitive average playtime; the completion times were evenly distributed across almost the entire spectrum, ranging from **20–30 minutes** to **more than one hour**, as seen in Figure 7.1. This indicates that the difficulty of the puzzles and the exploration of the System Layer scale very subjectively depending on the individual player.

Regarding the branching narrative, **two** players achieved the ending by helping K-P escape, while **three** allied with the Curator. Alongside the successful completion of the endings, **two** players failed to either help K-P or the Curator. This is visualized in Figure 7.2.

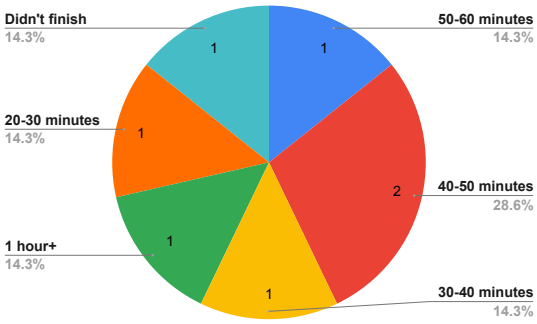


Figure 7.1: The graph showcasing the game testers' playtime.

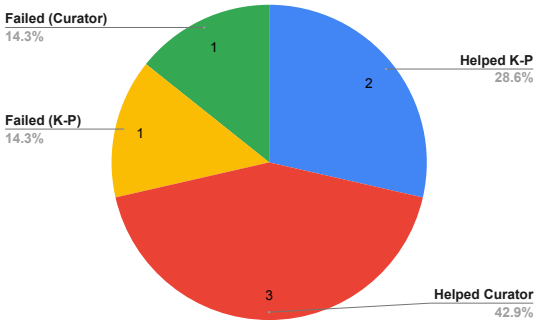


Figure 7.2: Overview of the narrative endings achieved by the playtesters.

User Experience and Clarity

The instructions of specific puzzles were generally considered clear or understandable, as seen in Figure 7.3. However, the playtesters noted that the instructions for finding a password for a specific encrypted file were not communicated clearly enough. This issue was remedied by hiding hints in the metadata of loaded files in the System Layer, as described in Section 5.2.1.

The playtesters were also asked about the intuitiveness of the System Layer. The responses in Figure 7.4 indicate that the user interface in the System Layer is highly intuitive.

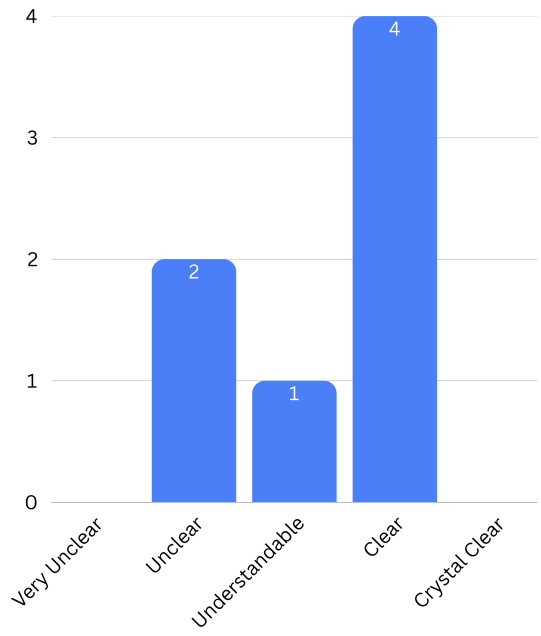


Figure 7.3: Playtesters' evaluation of the puzzle instructions' clarity.

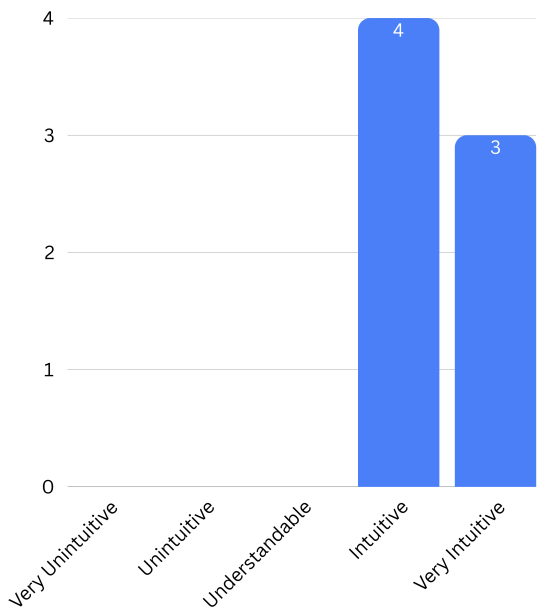


Figure 7.4: Playtesters' evaluation of the System Layer's UI intuitiveness.

Atmosphere and Fourth-Wall Breaks

To evaluate the effectiveness of the meta-horror elements, players were asked to rate their level of unease when the game interacted directly with their physical desktops. As shown in Figure 7.5, the playtesters reported a moderate level of apprehension on average. However, it is important to acknowledge a limitation in the testing methodology: for safety reasons, participants were informed in advance about certain intrusive mechanics. This prior knowledge likely mitigated the element of surprise and inevitably dampened the intended psychological impact.

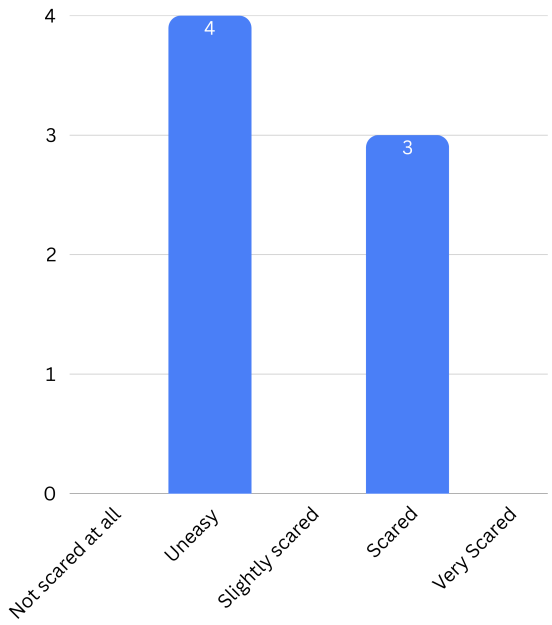


Figure 7.5: Playtesters' self-reported level of unease caused by the game's interaction with the host operating system.

Correspondingly, the testers were also asked to list their most unsettling moment during the gameplay. The most interesting answers include:

- Player 1: *'First crash'*
- Player 2: *'Creating files in my computer and detection of me deleting them'*
- Player 3: *'Slow spawning of Last/Warning/Leave/Me/Alone'*

Puzzles and Enjoyability

The overall difficulty of the puzzles was evaluated on a scale from *Too Easy* to *Too Hard*, and the results, displayed in Figure 7.6, suggest a well-balanced experience.

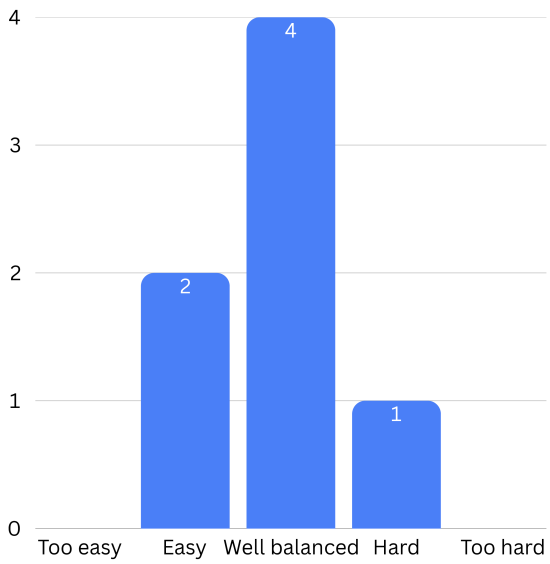


Figure 7.6: Playtesters' evaluation of the overall puzzle difficulty.

The playtesters were asked to grade each of the eight major puzzles on a scale from *Boring and Annoying* to *Fun and Interesting*. As the data shown in Figure 7.7 clearly indicates, the reception was overwhelmingly positive, with the vast majority of cumulative ratings falling into the *Interesting* or *Fun and Interesting* categories. While the graph shows an overall positive trend, the raw data indicate that deleting the corrupted files was consistently rated as the most enjoyable by the playtesters, whereas interacting with the external website was the least engaging.

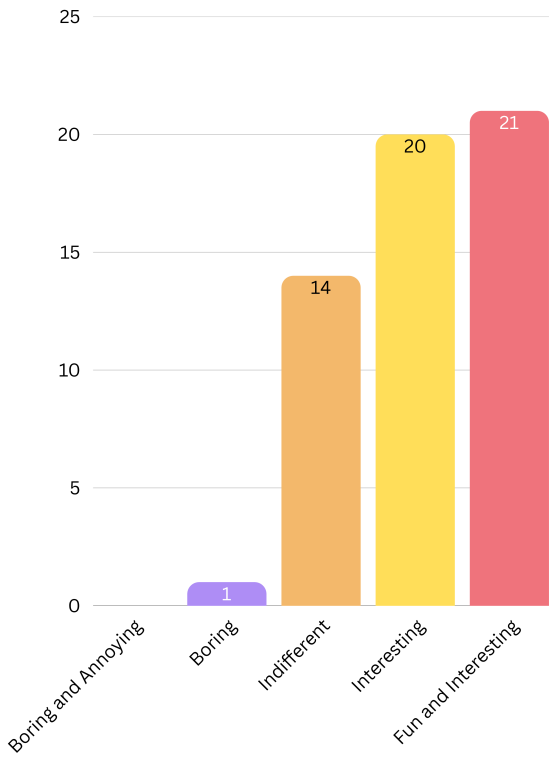


Figure 7.7: Aggregated playtester evaluations across all implemented puzzles regarding their entertainment value.

Debugging The testing also revealed multiple game-breaking bugs that could be triggered by performing specific, albeit intended, actions. Due to the nature of the game, many of these bugs corrupted the save file, effectively preventing the player from progressing the game's narrative. One specific instance of this bug involved the story manager not being subscribed to the event that advances the narrative to another state.

All of the aforementioned bugs and user feedback were logged and subsequently patched in the final release build. This feedback loop demonstrated the need for external testing, highlighting the gap between developer-intended interactions and actual user behavior.

Chapter 8

Conclusion

The primary objective of this bachelor's thesis was the design and technical implementation of *Kernel_Panic*, a meta-horror game that breaks the fourth wall by integrating the host operating system as a part of the game. By shifting the gameplay focus from a traditional simulated environment confined to the GUI's window to the player's actual desktop, the project explored the boundaries between software as a narrative tool and software as an active participant in reality.

While the Unity engine provided a robust foundation for the Simulated Layer's user interface, its inherent sandboxing required advanced software engineering to implement the System Layer's mechanics. The successful integration of Platform Invoke (P/Invoke) to communicate with native Windows APIs, the development of a custom C++ DLL for audio state monitoring, and the orchestration of external WPF applications demonstrated that a game engine can be effectively extended beyond its intended scope.

Furthermore, the inclusion of an external web-based ARG component facilitated unique interactions even beyond the user's computer. Architecturally, adopting a modified MVC pattern and strategically using asynchronous logic via Coroutines ensured that complex interactions remained performant and maintainable.

From a narrative and design perspective, it is important to reflect on the project's original vision. Initially, the gameplay was intended to lean more heavily on detective-style investigation, requiring the player to perform more extensive, methodical deduction. In retrospect, however, this investigative depth was not fully realized to the desired degree. The technical complexity of implementing the fourth-wall-breaking mechanics in the System Layer often took precedence during development, which occasionally overshadowed the more nuanced detective elements originally envisioned for the player.

The final result is a functional software prototype that achieves its goal of psychological immersion. *Kernel_Panic* forces the player to perceive their computer not as a neutral observer's platform, but as a compromised environment directly influenced by the game's narrative.

Future work could expand the System Layer's capabilities and explore cross-platform integration (such as support for Linux-based OS). Ultimately, this work serves as a proof of concept that technical limitations can be transformed into creative strengths through deliberate architectural design and low-level system engineering.

Bibliography

[1] AnthonyRox. *Message Notification 4* online. 2024. Available at: https://freesound.org/people/AnthonyRox/sounds/740423/. [cit. 18-April-2026].

[2] Dehairs, W. *Flexible Color Picker* online. 2022. Available at: https://assetstore.unity.com/packages/tools/gui/flexible-color-picker-150497. [cit. 04-April-2026].

[3] Gökçe, G. *Unity Standalone File Browser* online. 2018. Available at: https://github.com/gkngkc/UnityStandaloneFileBrowser. [cit. 13-April-2026].

[4] Huizinga, J. *Homo ludens*. 2nd ed. Prague: Dauphin, 2000. Studie. ISBN 80-727-2020-1.

[5] Knightley, S. *JSZip* online. 2025. Available at: https://github.com/Stuk/jszip. [cit. 13-April-2026].

[6] Microsoft Corporation. *.NET fundamentals documentation* online. N.d. Available at: https://learn.microsoft.com/en-us/dotnet/standard/. [cit. 12-April-2026].

[7] National Institute of Standards and Technology. *Secure Hash Standard (SHS)* online. 2015. Available at: https://doi.org/10.6028/NIST.FIPS.180-4. [cit. 13-April-2026].

[8] Newton King, J. *Json.NET* online. 2026. Available at: https://github.com/JamesNK/Newtonsoft.Json. [cit. 13-April-2026].

[9] OceanKing. *Lighthouse* online. 2024. Available at: https://pixabay.com/music/beats-lighthouse-219743/. [cit. 18-April-2026].

[10] Otuyama, J. M. *Stereogram Tutorial* online. N.d. Available at: https://www.ime.usp.br/~otuyama/stereogram/basic/index.html. [cit. 03-March-2026].

[11] Perron, B. and Barker, C. *Horror Video Games: Essays on the Fusion of Fear and Play*. 1st ed. Jefferson: McFarland & Company, Inc., 2009. ISBN 978-0-7864-4197-6.

[12] Pixel-cat-0. *Scary Face* online. 2022. Available at: https://www.pixilart.com/art/scary-face-4e414c36d5f9c5f. [cit. 18-April-2026].

[13] Reddy, G. R.; Reddy, P. L. V.; Priya, S. V.; Reddy, P. A. and Srinivas, K. Image Encryption and Decryption Using XOR operator. *Journal of Information Systems Engineering and Management* online, 2025, vol. 10, 13s. Available at: https://doi.org/10.52783/jisem.v10i13s.2027. [cit. 13-April-2026].

[14] Salen, K. and Zimmerman, E. *Rules of Play: Game Design Fundamentals*. 1st ed. Cambridge, Massachusetts: The MIT Press, 2004. ISBN 978-0-262-24045-1.

[15] Savage, J. E. *Models of Computation: Exploring the Power of Computing*. 1st ed. Reading, Mass.: Addison-Wesley, 1998. ISBN 978-0201895391. Chapter 10: Space-Time Tradeoffs.

[16] Schell, J. *The Art of Game Design: A Book of Lenses*. 3rd ed. Boca Raton: A K Peters/CRC Press, 2019. ISBN 9781315208435.

[17] Stallings, W. *Cryptography and Network Security: Principles and Practice, Global Edition*. 7th ed. Essex: Pearson, 2016. ISBN 9781292158587.

[18] stwime. *Freesound* online. 2020. Available at: https://freesound.org/people/stwime. [cit. 18-April-2026].

[19] xloizlol (Freesound). *Jumpscare* online. 2022. Available at: https://pixabay.com/sound-effects/horror-jumpscare-94984/. [cit. 18-April-2026].